

**Професионална гимназия по компютърни науки и математически
анализи
„Проф. Минко Балкански“ – Стара Загора**

ТЕОРЕТИЧНА ПИСМЕНА ЗАЩИТА

към практически проект по разработка на софтуер

на тема:

Enterprise платформа за организиране на турнири

(Tournament Platform)

Изготвил: Илия Димитров Влахов

12А клас

Стара Загора, 2026 г.

СЪДЪРЖАНИЕ

1. Увод	6
1.1. Актуалност на темата	6
1.2. Цел на проекта	6
1.3. Задачи за постигане на целта	6
1.4. Обект и предмет на разработката	7
1.5. Структура на писмената защита	7
2. Теоретична част	9
2.1. Същност на организирането на турнири	9
2.1.1. Формати на провеждане	9
2.1.2. Поставяне на участниците и свободни позиции (BYE)	10
2.1.3. Класиране и показатели при равенство	11
2.1.4. Резултати в реално време и известяване	11
2.2. Преглед на съществуващи решения	11
2.3. Технологичен стек	13
2.3.1. Език за програмиране Java 17	13
2.3.2. Spring Boot и Spring Web MVC	13
2.3.3. Spring Data JPA и Hibernate	13
2.3.4. Система за управление на бази данни MySQL	14
2.3.5. Spring Security	14
2.3.6. Валидация, Lombok и Maven	14
2.3.7. Клиентска част и инструменти за тестване	15
2.4. Архитектурни стилове	15
2.4.1. Клиент-сървър и REST	15
2.4.2. Слоеста архитектура	16
2.5. Обосновка на избора на технологии	16
3. Анализ и проектиране на системата	18
3.1. Анализ на функционалните изисквания	18
3.2. Анализ на нефункционалните изисквания	18
3.3. Роли на потребителите и права за достъп	19
3.4. Архитектура на системата	20
3.5. Проектиране на модела на данните	22
3.6. Случаи на употреба	24
3.7. Жизнен цикъл на турнира	26
3.8. Проектиране на REST интерфейса	27
4. Реализация на системата	28
4.1. Среда за разработка и използвани инструменти	28
4.2. Структура на проекта	28

4.3. Модул „Турнири“ и регистрация на участници	29
4.4. Модул „Схема“ – генериране на елиминационна схема.....	31
4.4.1. Размер на схемата и поставяне	32
4.4.2. Уреждане на мачовете и свободните позиции.....	33
4.4.3. Запис на схемата в базата данни	34
4.5. Модул „Схема“ – двойна елиминация.....	35
4.5.1. Номериране на кръговете и запис в базата данни	36
4.5.2. Свободни позиции и голям финал	37
4.6. Групова фаза – график „всеки срещу всеки“	39
4.7. Модул „Резултати“ – резултати, класиране и статистика	40
4.7.1. Въвеждане на резултат.....	40
4.7.2. Класиране	42
4.7.3. Статистика и публикуване на крайните резултати	43
4.8. Модул „Известия“	44
4.9. Функции за участници и администратори	45
4.10. Реализация на нефункционалните изисквания	47
4.10.1. Сигурност и ролеви достъп	47
4.10.2. Валидиране и обработка на грешки.....	48
4.10.3. Цялостност на данните и производителност	49
4.11. Потребителски интерфейс	50
5. Тестване, резултати и демонстрация	51
5.1. Подход към тестването	51
5.2. Резултати от автоматизираните тестове.....	51
5.3. Примерни тестови случаи.....	52
5.4. Сквозна проверка на работещото приложение.....	53
5.5. Открити и отстранени дефекти	54
5.6. Известни ограничения.....	55
6. Заключение.....	56
6.1. Постигнати резултати	56
6.2. Лични изводи от разработката	56
6.3. Насоки за бъдещо развитие	57
Използвана литература	58
Приложение А. Изходен код на проекта	61
A.1. Структура на проекта	61
A.2. BracketGenerator.java	61
A.3. DoubleEliminationGenerator.java	65
A.4. RoundRobinScheduler.java	72
A.5. RankingCalculator.java	73
A.6. SecurityConfig.java	76

Приложение Б. Допълнителни снимки на екрана	78
Приложение В. Стартиране на системата	86
В.1. Изисквания	86
В.2. Стартиране с една команда	86
В.3. Стартиране на двете части поотделно	86
В.4. Демонстрационни профили	87
В.5. Изпълнение на тестовете	87

1. Увод

1.1. Актуалност на темата

Турнирите са една от най-разпространените форми на организирано състезание – от училищни първенства по шах, тенис на маса и футбол до професионални лиги в електронните спортове. Независимо от дисциплината организаторът решава едни и същи задачи: събира заявките за участие, определя двойките за всеки кръг, записва резултатите, изчислява класирането и съобщава на участниците кога и срещу кого играят. Когато тези дейности се извършват на хартия или в електронни таблици, грешките са почти неизбежни – пропуснат участник, двама от най-силните състезатели, които се срещат още в първия кръг, неправилно разпределени свободни позиции или класиране, което не съответства на реално изиграните мачове.

Интересът към електронните спортове и към състезанията в училищна среда непрекъснато расте, а с него – броят на турнирите и на хората, които ги следят. Участниците очакват да виждат схемата и резултатите веднага след всеки мач, а организаторите се нуждаят от инструмент, който не допуска противоречиви данни. Съществуващите онлайн платформи решават част от тези проблеми, но са облачни услуги с общо предназначение, върху които училището или клубът нямат контрол. Затова е актуално разработването на самостоятелна платформа с ясно разделени роли, проверим алгоритъм за генериране на схеми и гарантирана цялост на резултатите – качества, характерни за софтуера от корпоративен (enterprise) клас.

1.2. Цел на проекта

Целта на проекта е да се разработи уеб базирана клиент-сървър платформа, която автоматизира организирането на спортни и електронни турнири – от регистрацията на участниците, през автоматичното генериране на елиминационна схема или групова фаза и въвеждането на резултати, до класирането, статистиката и известяването на участниците – при строго ролево разграничаване на правата на администратор, организатор и участник.

1.3. Задачи за постигане на целта

За постигането на целта са поставени следните задачи:

- 1) Да се проучат предметната област и съществуващите платформи за турнири и да се формулират функционалните и нефункционалните изисквания към системата.
- 2) Да се проектират архитектурата на системата, моделът на данните, програмният интерфейс и правата на ролите администратор, организатор и участник.
- 3) Да се реализира коректен алгоритъм за генериране на елиминационна схема с поставяне на участниците и свободни позиции, за автоматично придвижване на победителите и за съставяне на график „всеки срещу всеки“ за груповата фаза.
- 4) Да се реализират модулите за въвеждане на резултати, класиране, статистика и известия.
- 5) Да се осигурят сигурност, ролеви достъп и цялостност на данните и да се изгради уеб интерфейс за трите роли.
- 6) Да се провери системата чрез автоматизирани и сквозни тестове и да се документират резултатите.

1.4. Обект и предмет на разработката

Обект на разработката е процесът по организиране и провеждане на турнири по системата на директните елиминации и в групова фаза – регистрация на участниците, съставяне на двойките, провеждане на мачовете, отчитане на резултатите и определяне на крайното класиране.

Предмет на разработката е софтуерната платформа Tournament Platform, реализирана с Java 17, Spring Boot, Spring Data JPA, Spring Security и MySQL, и по-специално алгоритмите за поставяне на участниците, генериране на схемата, придвижване на победителите и изчисляване на класирането, както и механизмите за ролеви достъп и защита на цялостта на резултатите.

1.5. Структура на писмената защита

Писмената защита се състои от шест глави, списък на използваната литература и три приложения. След настоящия увод глава 2 „Теоретична част“ представя форматите на турнирите, правилата за поставяне и класиране, съществуващите платформи, използваните технологии и архитектурни стилове, както и обосновката на технологичния избор. Глава 3 „Анализ и проектиране на системата“ анализира изискванията и описва ролите, архитектурата, модела на данните, случаите на употреба, жизнения цикъл на турнира и програмния интерфейс.

Глава 4 „Реализация на системата“ разглежда средата за разработка, структурата на проекта и всеки функционален модул, като поставя акцент върху алгоритмите за схемата, графика на груповата фаза и класирането и върху реализацията на нефункционалните изисквания. Глава 5 „Тестване, резултати и демонстрация“ описва подхода към тестването, получените резултати, открити и отстранени дефекти и известните ограничения. Глава 6 „Заклучение“ обобщава постигнатото, личните изводи и насоките за развитие. В приложенията са включени структурата и ключови части от изходния код, допълнителни снимки на екрана и инструкции за стартиране.

2. Теоретична част

2.1. Същност на организирането на турнири

Турнирът е поредица от мачове между определен брой участници – отделни състезатели или отбори, – проведени по предварително известни правила, в резултат на които се определят победител и крайно класиране. Правилата за съставяне на двойките и за отпадане от състезанието се наричат формат или система на турнира. Изборът на формат зависи от броя на участниците, от наличното време и от това доколко е важно всеки участник да изиграе повече мачове.

2.1.1. Формати на провеждане

При **системата с директни елиминации** (single elimination) загубилият мач отпада, а победителят продължава в следващия кръг. Всеки мач елиминира точно един участник, затова при n участници се изиграват точно $n - 1$ мача, а броят на кръговете е равен на $\log_2(n)$, закръглен нагоре. Форматът е бърз и лесен за разбиране, но един неуспешен ден е достатъчен силен участник да отпадне още в началото.

При **двойната елиминация** участникът отпада едва след втората си загуба. Загубилите в основната схема преминават в схема на загубилите, чийто победител се среща с победителя в основната схема. Броят на мачовете приблизително се удвоява, а организацията е значително по-сложна.

При системата **„всеки срещу всеки“** (round robin) всеки участник играе срещу всички останали, т.е. се провеждат $n(n - 1)/2$ мача. Класирането е най-справедливо, защото не зависи от жребия, но при голям брой участници мачовете стават твърде много. Разпределението на срещите по кръгове така, че никой да не играе два пъти в един кръг, е стандартизирано чрез таблиците на Бергер, които се използват например в правилата на Международната шахматна федерация [1].

Швейцарската система провежда предварително определен брой кръгове, като във всеки кръг се срещат участници с еднакъв или близък брой точки. Тя позволява да бъдат класирани много участници с малко кръгове, но двойките могат да се определят едва след края на предходния кръг. Често се използва и **комбиниран формат** – групова фаза по системата „всеки срещу всеки“, последвана от директни елиминации между най-добрите от всяка група, какъвто се прилага на световните първенства по футбол. Основните формати са сравнени в Таблица 1.

Таблица 1. Сравнение на основните формати на турнири

Формат	Брой мачове при n участници	Предимства	Недостатъци
Директни елиминации	$n - 1$	Малко мачове, ясен финал, подходящ за много участници	Отпадане след една загуба, зависимост от жребия
Двойна елиминация	около $2n - 2$	Втори шанс след първата загуба	Около два пъти повече мачове, по-сложна схема
Всеки срещу всеки	$n(n - 1)/2$	Най-справедливо класиране, еднакъв брой мачове за всички	Броят на мачовете расте квадратично
Швейцарска система	$k \times n/2$ при k кръга	Много участници с малко кръгове	Двойките се определят кръг по кръг
Групи и елиминации	зависи от разпределението	Справедлива първа фаза и кратък финален етап	Изисква правила за класиране в групите

2.1.2. Поставяне на участниците и свободни позиции (BYE)

Когато участниците са с различна сила, мястото им в схемата силно влияе на крайния резултат. **Поставянето** (seeding) присвоява на всеки участник пореден номер според очакваната му сила – по рейтинг, по предходно класиране или по решение на организатора – и го разполага в схемата така, че най-силните да се срещнат възможно най-късно. Стандартната схема за 8 позиции подрежда двойките като 1–8, 4–5, 2–7 и 3–6: поставените под номер 1 и 2 могат да се срещнат едва на финала, а първите четирима – не по-рано от полуфиналите. Шуенк показва, че именно тази подредба удовлетворява естествените изисквания за справедливост на турнир с директни елиминации [2].

Схемата с директни елиминации изисква броят на позициите в първия кръг да бъде степен на двойката. Ако участниците са например 6, схемата се допълва до 8 позиции и две от тях остават празни. Участникът срещу празна позиция преминава в следващия кръг без мач – казва се, че получава **BYE**. Свободните позиции се дават на най-високо поставените участници като предимство за по-доброто им класиране. Правилното разпределение гарантира още, че празна позиция никога не се среща с друга празна позиция и че от втория кръг нататък всички мачове са между реални участници.

Типична грешка в наивно реализираните схеми е свободната позиция да бъде „пренесена“ в следващия кръг. Тогава участник, получил BYE, може да бъде обявен за служебен победител и в следващия мач, чийто противник още не е определен, а понякога – и за победител в целия турнир, без да е изиграл нито един мач. Затова

алгоритъмът трябва да различава празна позиция от позиция, чийто участник все още се определя в предходен мач.

2.1.3. Класиране и показатели при равенство

При групова фаза класирането се определя от броя на победите или от точките. Разпространена е схемата от футбола – 3 точки за победа, 1 за равенство и 0 за загуба. При равен брой победи или точки се използват допълнителни показатели: разлика между отбелязаните и допуснатите точки, брой отбелязани точки, резултат от прекия двубой или жребий. Важно е правилата да бъдат детерминирани – при едни и същи резултати класирането трябва винаги да бъде едно и също.

При директните елиминации основният критерий е етапът, до който участникът е достигнал: шампионът е пръв, финалистът – втори, отпадналите на полуфинал – трети и четвърти и т.н. Участниците, отпаднали в един и същ кръг, могат да бъдат подредени по допълнителни показатели като брой победи и разлика в резултата.

2.1.4. Резултати в реално време и известяване

Участниците и публиката очакват да видят промяната в схемата веднага след края на мача. В уеб приложенията това се постига по два начина. При **периодичното запитване** (polling) клиентът през кратък интервал изпраща заявка към сървъра и изобразява получените данни. При **изпращането от сървъра** (server push) – чрез WebSocket или Server-Sent Events – сървърът сам уведомява клиента за промяната. Периодичното запитване е по-лесно за реализация и работи с обикновения протокол HTTP, а изпращането от сървъра е по-ефективно при хиляди едновременни зрители.

Известията информират участника за събитията, които го засягат: кога и срещу кого е следващият му мач и какъв е крайният резултат от турнира. За да бъдат полезни, известията трябва да се създават автоматично при самото събитие, да достигат само до засегнатите потребители и да не се дублират.

2.2. Преглед на съществуващи решения

На пазара има утвърдени платформи за организиране на турнири. Прегледани са четири от най-популярните, които се използват както за спорт, така и за електронни спортове.

Challonge [3] е онлайн услуга за създаване и споделяне на турнирни схеми. Поддържа директни и двойни елиминации, системата „всеки срещу всеки“ и

швейцарска система, позволява поставяне на участниците и вграждане на схемата в други сайтове. Основните функции са безплатни, а разширените се предлагат срещу абонамент.

start.gg [4] (по-рано **smash.gg**) е платформа, насочена към общностите на видеоигрите и електронните спортове. Освен схеми тя предлага страници на събития, регистрация на участници, провеждане на няколко турнира в рамките на едно събитие и връзка с платформи за излъчване.

Toornament [5] е платформа за управление на турнири по спорт и електронни спортове, която съчетава различни формати и етапи в един турнир, публични страници и програмен интерфейс за връзка с външни системи.

Battlefy [6] е насочена предимно към електронните спортове и обединява организатори, отбори и играчи – регистрация, съобщаване на резултатите и публично класиране. Сравнението на четирите платформи с настоящия проект по критериите, важни за училище или клуб, е обобщено в Таблица 2.

Таблица 2. Сравнение на съществуващи платформи за турнири

Критерий	Challonge	start.gg	Toornament	Battlefy	Настоящият проект
Насоченост	Универсален генератор на схеми	Общности на видеоигрите	Спорт и електронни спортове	Електронни спортове	Училищни и клубни турнири
Формати	Елиминации, групи, швейцарска система	Елиминации, групи, швейцарска система	Различни формати и етапи	Елиминации и групови етапи	Директни елиминации, групи
Модел на ползване	Облачна услуга	Облачна услуга	Облачна услуга	Облачна услуга	Собствен сървър
Контрол върху данните	При доставчика	При доставчика	При доставчика	При доставчика	Изцяло при организацията
Изходен код	Затворен	Затворен	Затворен	Затворен	Достъпен за организацията

Прегледът показва, че съществуващите решения са зрели и богати на функции, но са облачни услуги с общо предназначение: данните се съхраняват при доставчика, правилата за достъп не могат да бъдат пригодени към конкретната организация, а алгоритмите им не могат да бъдат проверени. Настоящият проект не цели да ги замени изцяло. Той предлага самостоятелна платформа за училище или клуб с прозрачен и тестван алгоритъм, ролеви модел, съобразен със структурата на организацията, и пълен контрол върху данните.

2.3. Технологичен стек

2.3.1. Език за програмиране Java 17

Java е обектно ориентиран език със статично типизиране, чиито програми се компилират до байткод и се изпълняват от виртуалната машина JVM, което ги прави преносими между операционните системи. Версия 17 е издание с дългосрочна поддръжка (LTS) [7]. Проектът използва въведените в последните версии възможности: записите (records) за неизменими обекти за пренос на данни, изразите switch, текстовите блокове и потоците (Stream API). Строгото типизиране и богатата стандартна библиотека позволяват алгоритмите да бъдат записани ясно и да бъдат тествани изолирано.

2.3.2. Spring Boot u Spring Web MVC

Spring Framework е най-разпространената рамка за корпоративни приложения на Java. Нейна основа е инверсията на управлението: обектите (bean-ове) не създават зависимостите си сами, а ги получават от контейнера чрез инжектиране на зависимости. Това намалява свързаността между класовете и улеснява тестването. Spring Boot [8] надгражда рамката с автоматична конфигурация, готови групи от зависимости (starters) и вграден уеб сървър Apache Tomcat, така че приложението се стартира като самостоятелен изпълним JAR файл.

Spring Web MVC [9] обработва HTTP заявките чрез централен диспечер (DispatcherServlet), който ги насочва към методи на контролерите. Класовете с анотация @RestController връщат директно данни, които библиотеката Jackson преобразува във формат JSON. Анотациите @GetMapping, @PostMapping, @PutMapping и @DeleteMapping свързват адресите и HTTP методите с конкретни операции.

2.3.3. Spring Data JPA u Hibernate

Обектно-релационното съпоставяне (ORM) свързва класовете на приложението с таблиците на релационната база данни. Стандартът Jakarta Persistence (JPA) [10] определя анотации като @Entity, @ManyToOne и @OneToMany, а Hibernate [11] е най-използваната му реализация. Hibernate генерира SQL заявките, пази заредените обекти в контекста на транзакцията и следи за промените в тях, които записва автоматично при потвърждаване на транзакцията.

Spring Data JPA [12] премахва повтарящия се код за достъп до данни: достатъчно е да се декларира интерфейс, наследяващ JpaRepository, а заявките се извеждат от

имената на методите или се задават на езика JPQL. Графите на обекти (@EntityGraph) указват кои свързани данни да се заредят с една заявка, а полето с анотация @Version реализира оптимистично заключване – ако двама потребители променят един и същ запис едновременно, втората промяна се отхвърля, вместо да презапише първата.

2.3.4. Система за управление на бази данни MySQL

MySQL е релационна система за управление на бази данни с отворен код [13]. Подсистемата за съхранение InnoDB поддържа транзакции със свойствата ACID – атомарност, консистентност, изолация и трайност, – външни ключове и заключване на ниво ред, а от версия 8.0.16 MySQL проверява и ограниченията CHECK. Тези възможности са съществени за турнирната платформа: резултатът от мача, придвижването на победителя и известията се записват в една транзакция, а уникалните ключове и проверките в базата данни пазят данните дори при грешка в приложението.

2.3.5. Spring Security

Spring Security [14] е рамка за автентикация (установяване на самоличността) и авторизация (проверка на правата) в Spring приложенията. Всяка HTTP заявка преминава през верига от филтри, които извличат данните за вход, проверяват ги чрез услуга от тип UserDetailsService и запазват удостоверения потребител заедно с ролите му. Правата се задават както за адреси, така и директно върху методи чрез анотацията @PreAuthorize.

Паролите не се съхраняват в явен вид, а като хеш, изчислен с алгоритъма BCrypt [15]. BCrypt добавя случайна „сол“ към всяка парола и има настройваема цена на изчислението, което прави атаките с предварително изчислени таблици и с груба сила бавни и скъпи. За пренос на данните за вход се използва схемата HTTP Basic [16]: клиентът изпраща потребителското име и паролата в заглавката Authorization, кодирани с Base64. Схемата е проста и не изисква сесия на сървъра, но в реална среда задължително трябва да се използва криптирана връзка (HTTPS).

2.3.6. Валидация, Lombok и Maven

Стандартът Jakarta Bean Validation [17] позволява ограниченията върху входните данни да се декларират с анотации върху полетата – @NotBlank, @Size, @Email, @Min, @FutureOrPresent – и да се проверяват автоматично, преди заявката да достигне бизнес логиката. Библиотеката Lombok [18] генерира по време на компилация рутинния

код – методи за достъп, конструктори и регистратори, – а Apache Maven [19] управлява зависимостите, компилирането, тестовите и пакетирането. Обвивката Maven Wrapper изтегля точно определена версия на Maven, така че проектът се изгражда еднакво на всеки компютър.

2.3.7. Клиентска част и инструменти за тестване

Клиентската част е реализирана с HTML5, CSS3 и JavaScript без допълнителна рамка. Съвременните браузъри предоставят всичко необходимо за приложение с една страница: Fetch API [20] за асинхронни HTTP заявки, шаблонни низове за генериране на разметка, CSS Grid и Flexbox за адаптивно оформление. Липсата на стъпка за компилиране прави клиента лесен за стартиране и за разглеждане.

За автоматизираните тестове се използват JUnit 5 [21] – стандартната рамка за модулни тестове в Java, включително параметризирани тестове, – и Mockito [22] за заместване на зависимостите с контролирани обекти. Spring MockMvc изпълнява HTTP заявки към контролерите без реален сървър, а Playwright [23] управлява истински браузър Chromium за проверка на потребителския интерфейс.

2.4. Архитектурни стилове

2.4.1. Клиент-сървър и REST

При архитектурата клиент-сървър потребителският интерфейс (клиентът) и обработката и съхранението на данните (сървърът) са отделни компоненти, които комуникират по мрежата. Така един сървър може да обслужва различни клиенти – уеб браузър, мобилно приложение или друга система, – а двете части могат да се развиват независимо.

REST (Representational State Transfer) е архитектурен стил за разпределени системи, описан от Рой Филдинг [24]. Основните му принципи са: ресурсите се идентифицират с адреси, например `/api/tournaments/5`; използва се единен интерфейс, в който HTTP методите изразяват действието – GET чете, POST създава, PUT променя, DELETE изтрива; комуникацията е без състояние, т.е. всяка заявка съдържа цялата информация, нужна за обработката ѝ; резултатът се съобщава чрез стандартни кодове на състояние като 200, 201, 400, 401, 403, 404 и 409.

2.4.2. Слоеста архитектура

Слоестата архитектура разделя приложението на слоеве с ясно определени отговорности, всеки от които използва само слоя под себе си [25]. В корпоративните Java приложения обикновено се обособяват: слой за представяне (REST контролери), който приема и валидира заявките; слой на бизнес логиката (услуги), който съдържа правилата на предметната област и управлява транзакциите; и слой за достъп до данни (хранилища), който скрива работата с базата данни. Обектите за пренос на данни (DTO) отделят формата на програмния интерфейс от вътрешния модел, така че промяна в базата данни да не променя договора с клиента.

Моделът MVC (Model–View–Controller) е приложен в разпределен вариант: контролерите на сървъра приемат заявките и връщат модела под формата на JSON, а изгледът се изгражда от клиента в браузъра.

2.5. Обосновка на избора на технологии

Технологичният стек Java 17, Spring Boot, Spring Data JPA, MySQL и Spring Security е зададен в заданието на проекта, но той е и естествен избор за система с изискванията на турнирна платформа:

- **Коректност на алгоритмите.** Статичното типизиране на Java и отделените компоненти на Spring позволяват алгоритъмът за схемата да бъде тестван с хиляди генерирани сценарии без база данни и без сървър.
- **Ролеви достъп.** Spring Security предлага готови и проверени механизми за автентикация, хеширане на пароли и декларативна авторизация, вместо те да бъдат писани наново.
- **Цялостност на резултатите.** Декларативните транзакции на Spring, оптимистичното заключване на JPA и ограниченията на MySQL гарантират, че резултатът, придвижването на победителя и известията се записват заедно или изобщо не се записват.
- **Естествен модел на данните.** Турнирите, участниците, групите и мачовете образуват ясно свързани таблици, за които релационната база данни е най-подходяща.
- **Преносимост и поддръжка.** Приложението работи на всяка операционна система с JDK 17 и MySQL, а слоестата структура улеснява добавянето на нови формати и функции.

- **Лесен клиент.** Клиентът без рамка не изисква компилиране и показва, че програмният интерфейс може да се използва от произволен клиент.

3. Анализ и проектиране на системата

3.1. Анализ на функционалните изисквания

Функционалните изисквания произтичат от заданието на проекта, което определя четири модула – турнири, схема, резултати и известия. В Таблица 3 всяко изискване от заданието е свързано с функциите, чрез които е реализирано. Освен задължителните функции са добавени и такива, които правят трите роли пълноценни: самостоятелно записване на участниците, страница „Моите мачове“, административно управление на потребителите и демонстрационни данни за представяне на системата.

Таблица 3. *Съответствие между заданието и реализираните функции*

Модул	Изискване от заданието	Реализация в системата
Турнири	Създаване на турнир с формат (директни елиминации, двойна елиминация или групи)	Организаторът създава, редактира и изтрива турнир с име, описание, формат и период; жизнен цикъл „Регистрация“ → „В ход“ → „Приключил“
Турнири	Регистрация на отбори и участници	Организаторът добавя участници или отбори и по желание ги свързва с потребителски профил; участникът се записва и отказва сам, докато регистрацията е отворена; групи с ръчно разпределение
Схема	Автоматично генериране на елиминационна схема	Схема с размер следващата степен на двойката, стандартно поставяне по реда на регистрацията, ВУЕ за най-високо поставените; за групи – график „всеки срещу всеки“
Схема	Автоматично напредване на победителя към следващия кръг	При запис на резултат победителят се поставя в правилната позиция на следващия кръг, а мачовете с двама известни участници стават готови за игра
Резултати	Въвеждане на резултати от мачове от организатор	Само собственикът на турнира или администратор; валидиране на резултата; забрана за равен резултат при елиминации
Резултати	Класиране и статистика на участниците	Класиране (изиграни мачове, победи, равни, загуби, разлика, точки), шампион и подиум, участник с най-много точки, най-голяма победа, напредък
Известия	Известия за предстоящи мачове	Автоматично известие до двамата участници и до организатора, щом двойката на мача е определена
Известия	Публикуване на крайни резултати	Известие до организатора и всички свързани участници при приключване; раздел „Резултати и статистика“

3.2. Анализ на нефункционалните изисквания

Заданието поставя три нефункционални изисквания, които в най-голяма степен определят качеството на системата:

- **Коректност на алгоритъма за схемата** – при всеки брой участници схемата трябва да съдържа правилния брой мачове и кръгове, свободните позиции да се дават на най-високо поставените участници, нито един мач да не бъде решен, преди да са известни двамата му участници, а турнирът да завършва с точно един шампион след $n - 1$ изиграни мача.
- **Ролеви достъп** – всяко действие е разрешено само на определените роли, а организаторът може да променя единствено собствените си турнири; проверките се извършват на сървъра, независимо от интерфейса.
- **Цялостност на резултатите** – резултат може да бъде въведен само за мач, готов за игра, само веднъж и само с валидни стойности; записът на резултата и последиците от него са атомарни, а едновременните промени не се презаписват.

Към тях се добавят общите изисквания към съвременен уеб приложение: сигурност на данните за вход и защита от неоторизиран достъп, съобразени с препоръките на OWASP [26]; валидиране на всички входни данни и еднакъв формат на грешките; използваемост и адаптивен интерфейс за компютър, таблет и телефон; лесна поддръжка и разширяемост чрез слоеста архитектура; производителност без излишни заявки към базата данни; преносимост между операционните системи.

3.3. Роли на потребителите и права за достъп

Системата разграничава три роли. **Участникът** разглежда турнирите, схемите и класиранията, записва се в турнири с отворена регистрация и следи собствените си мачове и известия. **Организаторът** създава турнири и ги управлява изцяло – участници, групи, генериране на схема и въвеждане на резултати, – но само за турнирите, които сам е създал. **Администраторът** има правата на организатор за всеки турнир и управлява профилите – ролите и блокирането на достъпа.

При регистрация потребителят избира роля участник или организатор. Роля администратор не може да бъде избрана публично, защото това би позволило на всеки да придобие пълни права. Първият администратор се създава при стартиране от конфигурацията на сървъра, а следващите се назначават от съществуващ администратор. Правата на ролите са обобщени в Таблица 4.

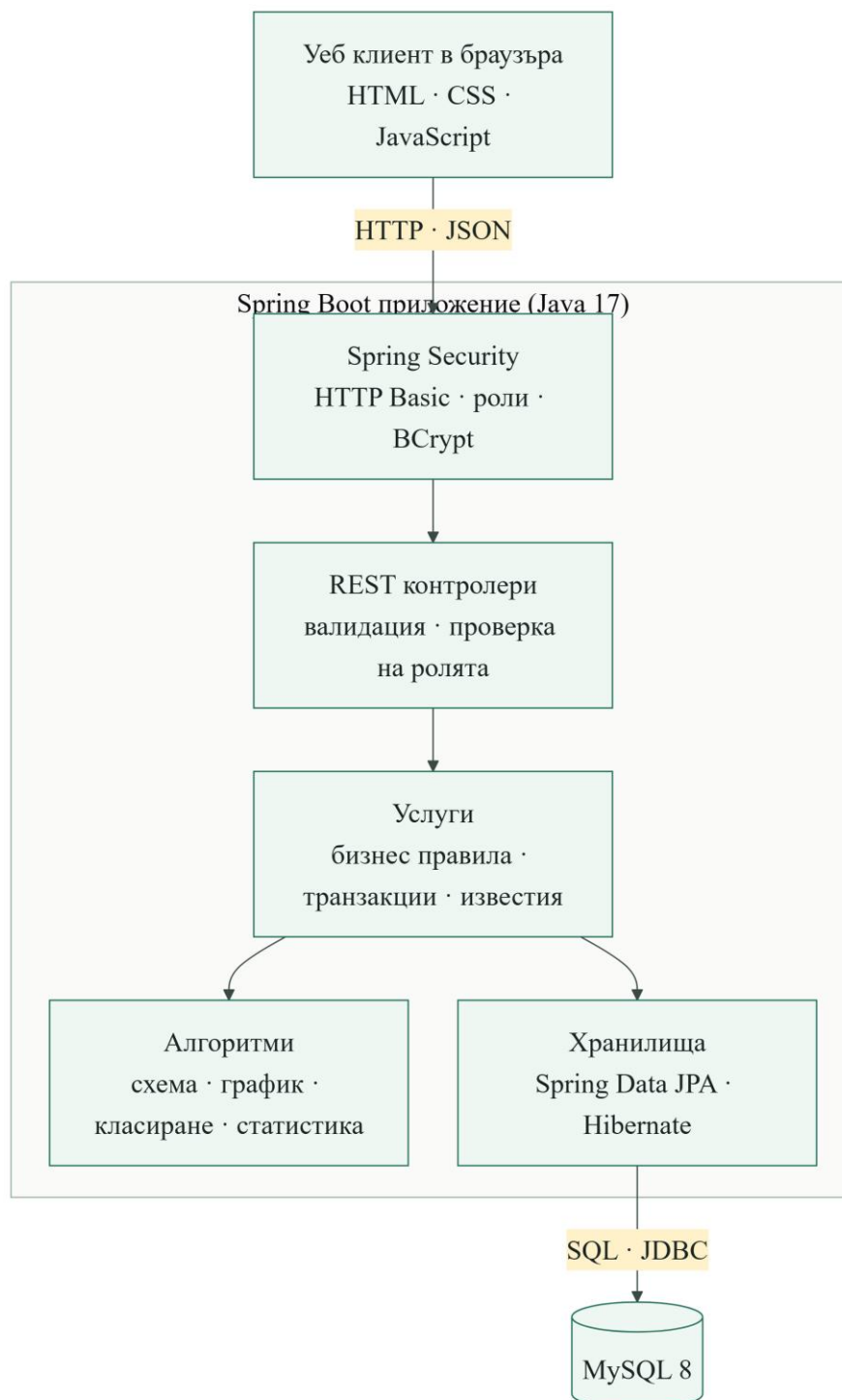
Таблица 4. Права на ролите в системата

Действие	Участник	Организатор	Администратор
Преглед на турнири, схеми, класиране, резултати и статистика	Да	Да	Да

Действие	Участник	Организатор	Администратор
Създаване на турнир	Не	Да	Да
Редакция и изтриване на турнир в регистрация	Не	Само собствени	Всички
Добавяне на участници и управление на групи	Не	Само собствени	Всички
Записване и отказ от турнир	Да	Не	Не
Генериране на схема или групови мачове	Не	Само собствени	Всички
Въвеждане на резултат	Не	Само собствени	Всички
Страница „Моите мачове“	Да	Не	Не
Собствени известия	Да	Да	Да
Управление на потребители и роли	Не	Не	Да, без собствения профил

3.4. Архитектура на системата

Системата е изградена по архитектурата клиент-сървър с REST интерфейс, а сървърът – по слоестия модел. Общата архитектура е показана на Фигура 1.



Фигура 1. Архитектура на системата

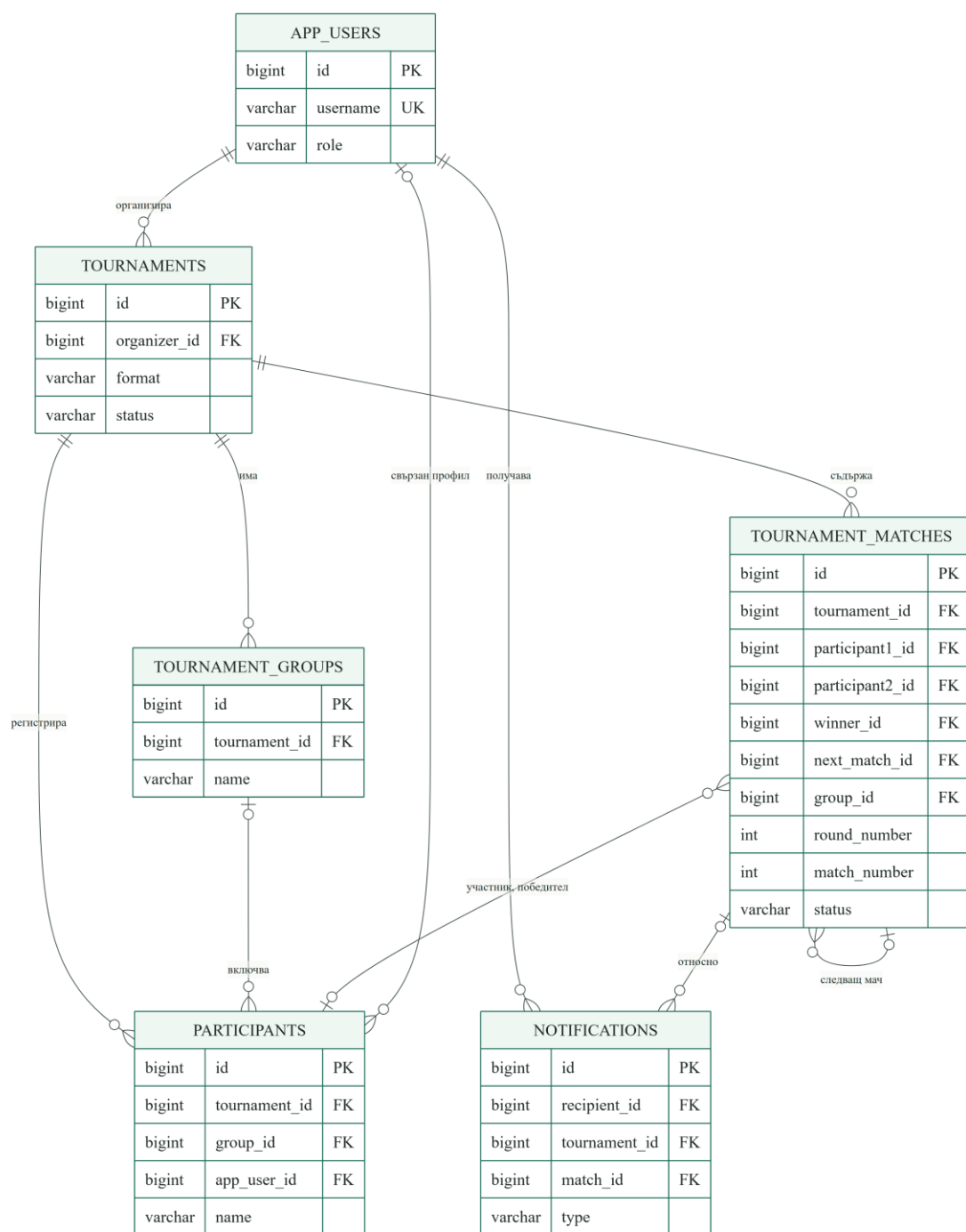
Уеб клиентът се изпълнява в брауъра и комуникира със сървъра само чрез HTTP заявки с данни във формат JSON. Всяка заявка първо преминава през веригата от филтри на Spring Security, която установява потребителя по заглавката Authorization и отказва достъп до защитените адреси без валидни данни. Контролерите приемат заявката, валидират тялото ѝ и проверяват ролята. Услугите изпълняват бизнес

правилата – собственост върху турнира, допустим статус, уникалност – в рамките на транзакция и използват отделни компоненти за алгоритмите: генериране на елиминационна схема, график „всеки срещу всеки“, класиране и статистика. Хранилищата на Spring Data JPA превръщат операциите в SQL заявки към MySQL.

Алгоритмите са поставени в самостоятелни компоненти без достъп до базата данни. Така едни и същи класове се използват от услугите в работещото приложение и директно от тестовете, което е ключово за доказване на тяхната коректност.

3.5. Проектиране на модела на данните

Моделът на данните се състои от шест таблици. Връзките между тях са представени на ER диаграмата на Фигура 2, а предназначението на всяка таблица е описано в Таблица 5.



Фигура 2. ER диаграма на базата данни

Таблица 5. Таблицы в базата данни

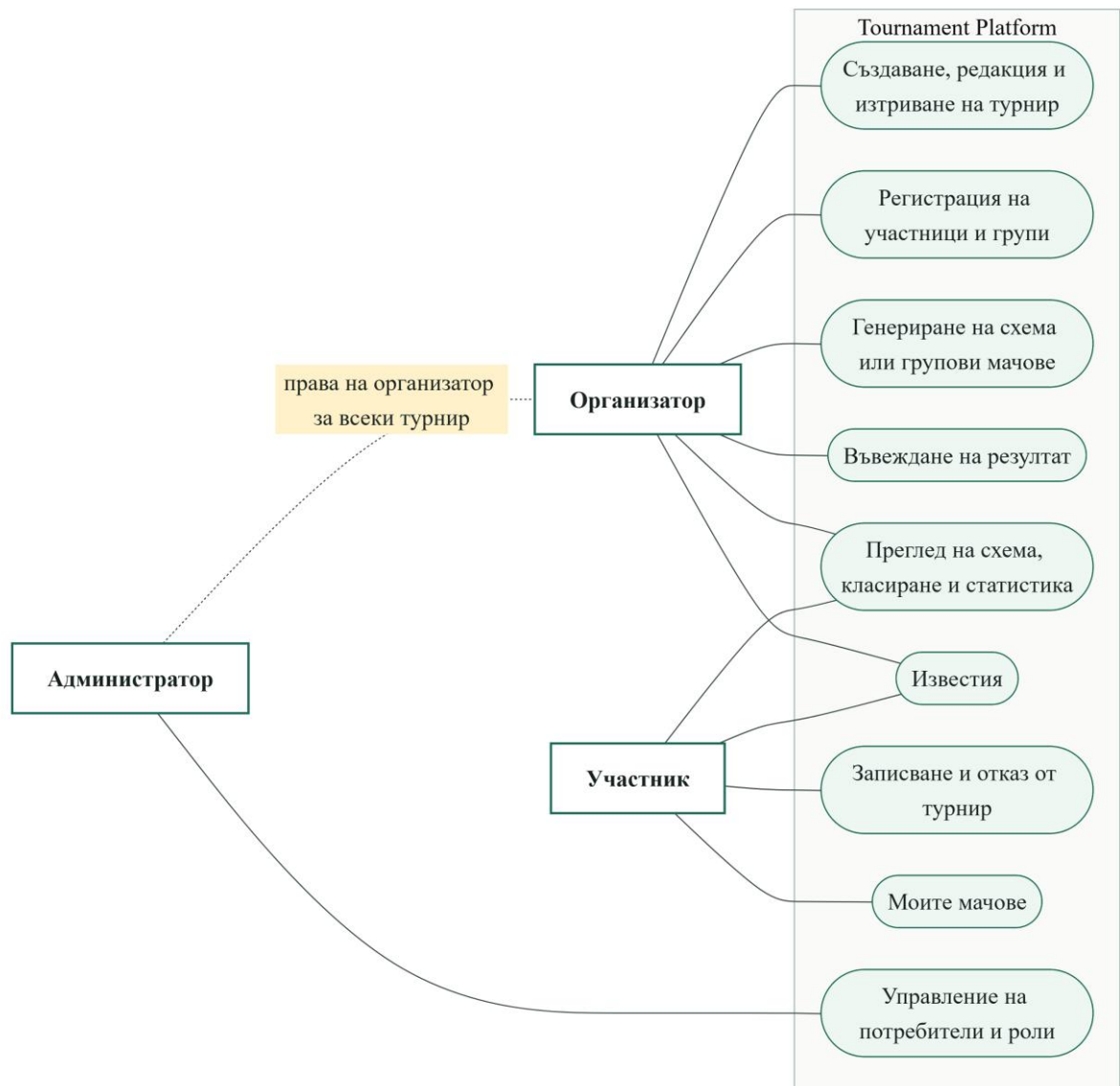
Таблица	Предназначение	Важни ограничения
app_users	Потребителски профили: име, имейл, хеш на паролата, роля, активност	Уникални потребителско име и имейл
tournaments	Турнири: име, описание, формат, статус, период, организатор, правило за реванш	Проверка „край $\geq$ начало“; версия за оптимистично заключване

Таблица	Предназначение	Важни ограничения
participants	Участници или отбори в конкретен турнир, група и свързан профил	Уникално име в турнира без значение от регистъра на буквите
tournament_groups	Групи на турнир с групов фаза	Уникално име на групата в турнира
tournament_matches	Мачове: кръг, номер, участници, резултат, победител, статус, следващ мач, група	Уникална позиция (турнир, кръг, номер); неотрицателни резултати; версия
notifications	Известия: получател, вид, текст, прочетено, турнир и мач	Индекс по получател, прочитане и дата

При проектирането са взети няколко решения, които пряко обслужват нефункционалните изисквания. Участникът принадлежи на точно един турнир, а името му се пази и в нормализиран вид с малки букви; уникалният ключ върху турнира и нормализираното име предотвратява дублиране дори при едновременни заявки. Всеки мач сочи към следващия мач в схемата чрез външен ключ към същата таблица (`next_match_id`), което описва дървовидната структура на елиминационната схема. Полетата за участниците в мача допускат празна стойност – така се представят свободна позиция (BYE) и участник, който още не е определен. Таблиците на турнирите и мачовете имат колона `version`, която Hibernate използва за оптимистично заключване, а резултатите и датите се проверяват и чрез ограничения CHECK в базата данни.

3.6. Случаи на употреба

Основните сценарии, в които участват трите роли, са показани на Фигура 3.

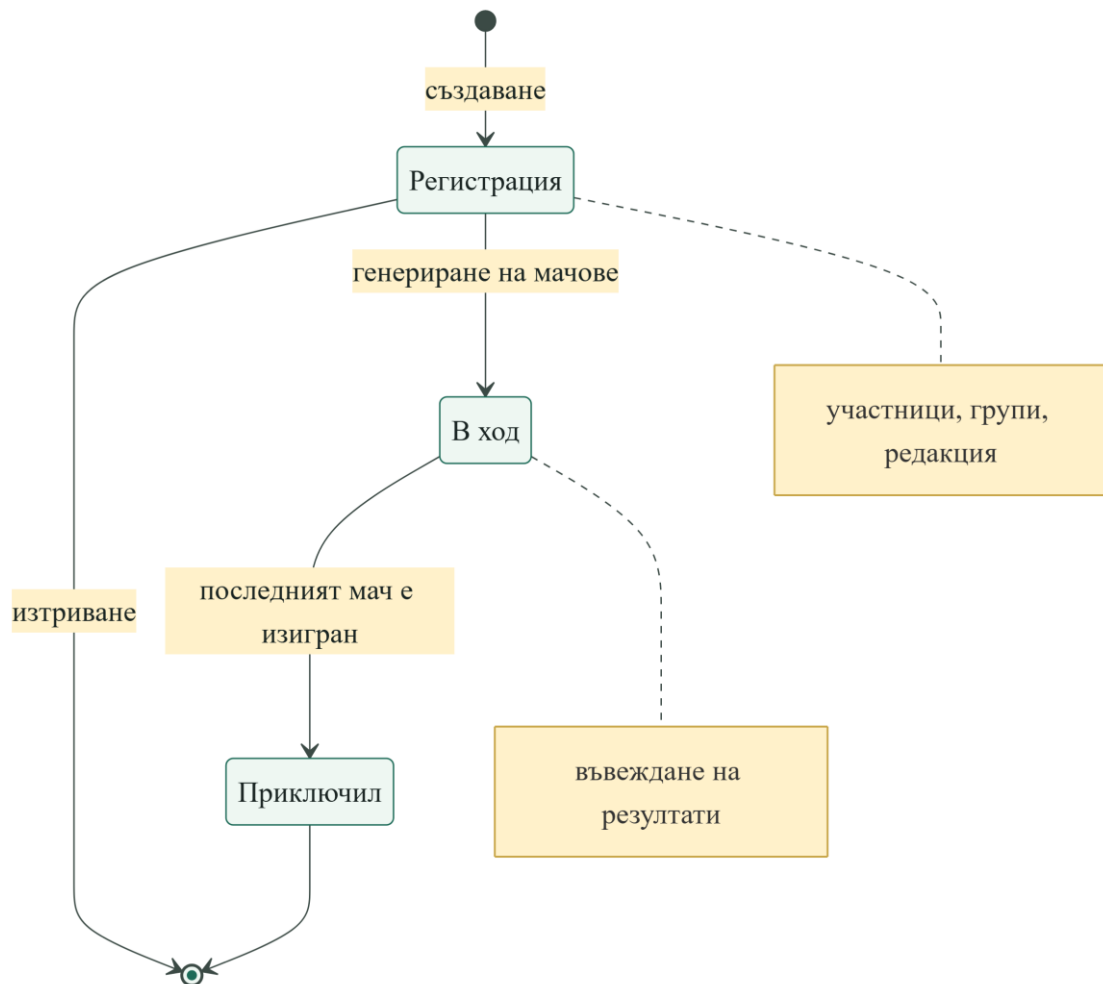


Фигура 3. Диаграма на случаите на употреба

Типичният сценарий започва с организатор, който създава турнир и добавя участниците или изчаква те да се запишат сами. При групов формат организаторът създава групи и разпределя участниците в тях. След затваряне на регистрацията той генерира схемата, при което системата изпраща известия за първите мачове. След всеки изигран мач организаторът въвежда резултата, победителят автоматично преминава напред, а участниците в новоопределените мачове получават известие. След последния мач турнирът приключва и крайното класиране се публикува. Участниците следят схемата в реално време и собствените си мачове, а администраторът поддържа потребителските профили.

3.7. Жизнен цикъл на турнира

Турнирът преминава през трите състояния, показани на Фигура 4. Допустимите действия зависят от текущото състояние, което е основна защита на цялостта на данните.



Фигура 4. Жизнен цикъл на турнир

- **Регистрация (REGISTRATION)** – турнирът може да бъде редактиран и изтрит; добавят се и се премахват участници и групи. Мачове все още не съществуват.
- **В ход (IN_PROGRESS)** – настъпва при генериране на схемата или на груповите мачове. Регистрацията е затворена и се въвеждат резултати. Турнир в това състояние не може да бъде изтрит, защото вече съдържа мачове и резултати.
- **Приключил (COMPLETED)** – настъпва автоматично, когато всички мачове са приключени. Резултатите вече не могат да бъдат променяни, а шампионът и класирането се публикуват.

3.8. Проектиране на REST интерфейса

Програмният интерфейс следва принципите на REST: ресурсите са турнири, участници, групи, мачове, известия и потребители, а действията се изразяват чрез HTTP методи. Основните адреси са дадени в Таблица 6.

Таблица 6. Основни адреси на REST интерфейса

Метод и адрес	Предназначение	Достъп
POST /api/auth/register POST /api/auth/login	Регистрация и вход	Публичен
GET /api/tournaments GET /api/tournaments/{id}	Списък и детайли на турнирите	Вписан потребител
POST /api/tournaments	Създаване на турнир	Организатор, администратор
PUT /api/tournaments/{id} DELETE /api/tournaments/{id}	Редакция и изтриване	Собственик, администратор
GET, POST /api/tournaments/{id}/participants	Преглед и добавяне на участници	Вписан потребител / собственик
POST, DELETE /api/tournaments/{id}/join	Записване и отказ от турнир	Участник
/api/tournaments/{id}/groups/...	Групи и разпределение на участниците	Вписан потребител / собственик
POST /api/tournaments/{id}/generate-bracket	Генериране на схема или групови мачове	Собственик, администратор
GET /api/tournaments/{id}/bracket .../rankings, .../results, .../statistics	Схема, класиране, резултати и статистика	Вписан потребител
POST /api/matches/{id}/result	Въвеждане на резултат	Собственик, администратор
GET /api/matches/mine	Мачовете на текущия потребител	Вписан потребител
GET /api/notifications PUT .../{id}/read, PUT .../read-all	Известия и отбелязване като прочетени	Получателят
GET /api/admin/overview GET /api/admin/users	Обобщение на платформата и списък с потребители	Администратор
PUT /api/admin/users/{id}/role PUT /api/admin/users/{id}/status	Смяна на роля, блокиране	Администратор

Отговорите са във формат JSON. Успешното създаване връща код 201, а изтриването – 204. Грешките имат еднаква структура с полетата `timestamp`, `status`, `error`, `message` и `path`, а кодът на състояние показва вида им: 400 при невалидни данни или нарушено бизнес правило, 401 при липсваща автентикация, 403 при недостатъчни права, 404 при несъществуващ ресурс и 409 при конфликт, например дублирано име или едновременно промяна. Съобщенията никога не съдържат вътрешна информация като стек на извикванията или SQL заявки.

4. Реализация на системата

4.1. Среда за разработка и използвани инструменти

Проектът е разработен и изпитан под операционна система Windows 11. Използваните инструменти и техните версии са обобщени в Таблица 7.

Таблица 7. Среда за разработка и инструменти

Инструмент	Версия	Предназначение
Visual Studio Code с Extension Pack for Java	–	Редактиране, навигация в кода и дебъгване
Eclipse Temurin JDK	17.0.20	Компилиране и изпълнение на Java кода
Spring Boot	3.4.5	Рамка на приложението (Spring Framework 6.2, Hibernate ORM 6.6, Apache Tomcat 10.1)
Apache Maven чрез Maven Wrapper	3.9.9	Зависимости, компилиране, тестове, изграждане на JAR файл
MySQL Server и MySQL Workbench	8.0.46	База данни и графичен клиент за нея
Lombok	1.18.46	Генериране на рутинен код
JUnit 5, Mockito, Spring Test	според Spring Boot 3.4.5	Автоматизирани тестове
Node.js и Playwright (Chromium)	Node.js 24	Стартиране с една команда, статичен сървър за клиента, проверки и снимки в браузър

Исходният код е организиран като самостоятелен Maven проект и се предава като архив заедно с инструкциите за стартиране във файла README.md. Използването на системата за контрол на версиите Git с отдалечено хранилище е предвидено като следваща стъпка в развитието на проекта (вж. раздел 6.3).

Настройките, които зависят от конкретния компютър – потребителското име и паролата за MySQL, – не са част от изходния код. Приложението ги чете от локален файл `application-local.properties` или от променливи на средата, а в проекта е включен само примерен файл без пароли. Така чувствителните данни не се разпространяват заедно с кода.

4.2. Структура на проекта

Сървърната част следва стандартната структура на Maven проект, а Java пакетите съответстват на слоевете от архитектурата:

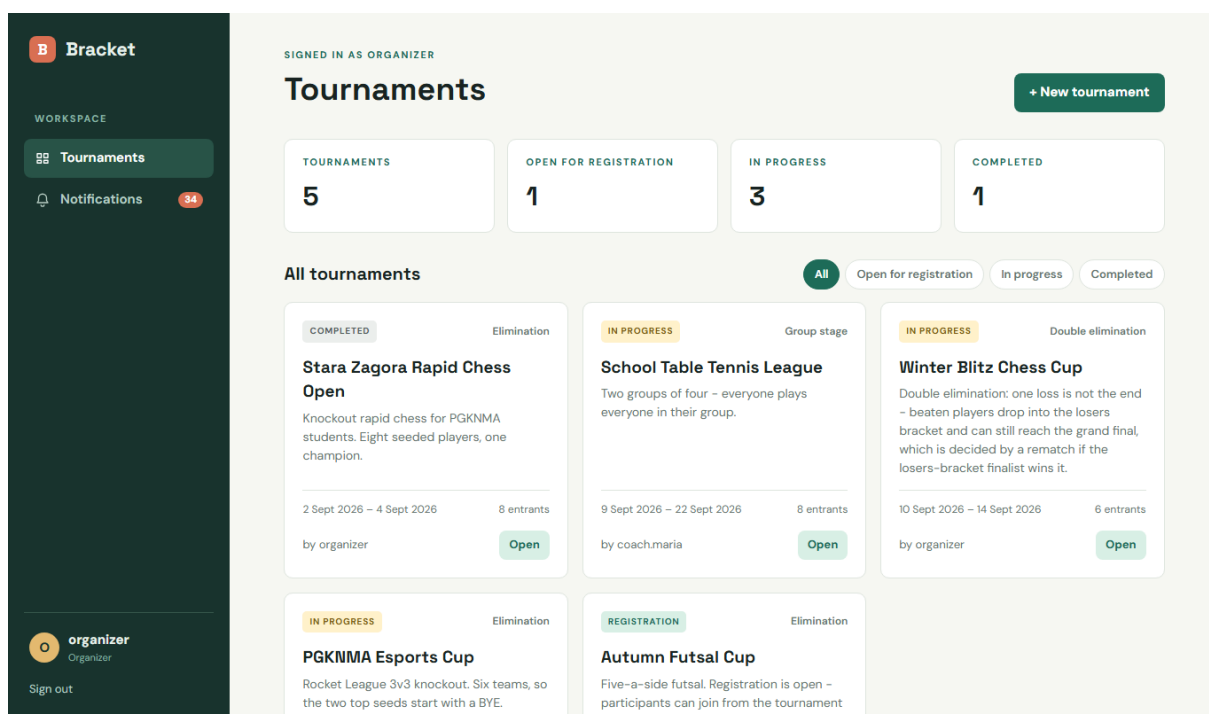
```
src/main/java/com/ilko/tournament/
├── config/      стартиране: проверка на схемата, първи администратор,
│               демонстрационни данни
├── controller/ REST контролери (auth, tournaments, matches, notifications,
│               admin)
```

dto/валидация	обекти за заявки и отговори (Java records) с правила за валидация
entity/	JPA обекти: AppUser, Tournament, Participant, TournamentGroup, TournamentMatch, Notification
enums/	роли, формати и статуси
exception/	изключения и глобален обработчик на грешки
mapper/	преобразуване на обекти в DTO
repository/	хранилища на Spring Data JPA
security/	конфигурация на Spring Security и зареждане на потребители
service/	интерфейси на услугите и алгоритмите
impl/	реализации на услугите
src/main/resources/	application.properties, application-prod.properties
src/test/java/	модулни, алгоритмични, сигурностни и интеграционни тестове
frontend/	index.html, styles.css, app.js, server.js
e2e/	сквозни проверки и снимки на екрана
docs/	бизнес правила, стратегия за тестване, миграции
dev.js, package.json	стартиране на сървъра и клиента с една команда

Всеки пакет има една отговорност. Контролерите не съдържат бизнес логика, а само свързват HTTP адресите с методите на услугите; услугите не зависят от HTTP; алгоритмите не зависят нито от HTTP, нито от базата данни. Клиентът е отделен в папката frontend и може да бъде обслужван от произволен статичен уеб сървър.

4.3. Модул „Турнири“ и регистрация на участници

Турнирът е представен от обекта Tournament с формат (ELIMINATION, DOUBLE_ELIMINATION или GROUPS), статус и връзка към организатора. При създаване статусът автоматично е „Регистрация“, а за организатор се записва вписаният потребител. Началният екран със списъка на турнирите, обобщението и филтрите по статус е показан на Фигура 5, а формата за създаване на турнир – на Фигура 20 в Приложение Б.



Фигура 5. Списък с турнири от гледна точка на организатор

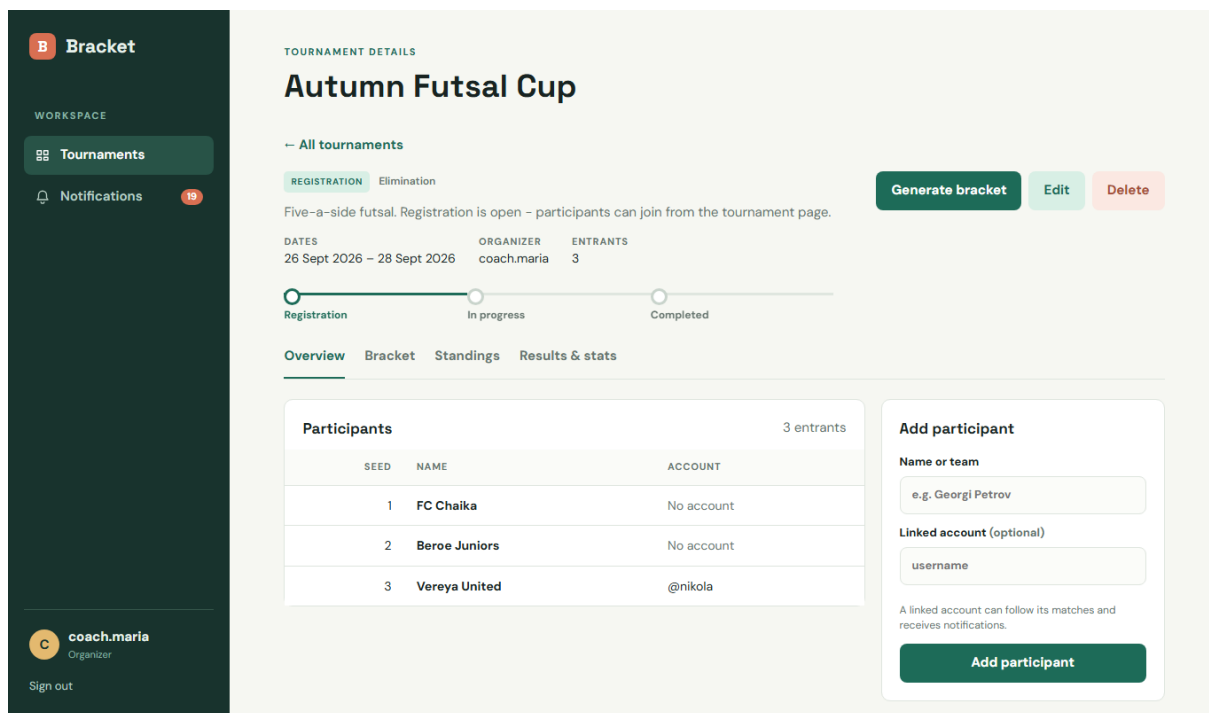
Проверката на правата е на две нива. Контролерът разрешава метода само за роли организатор и администратор, а услугата проверява собствеността върху конкретния турнир преди всяка промяна. Администраторът преминава проверката винаги, а организаторът – само за собствените си турнири, така че дори при директна заявка към програмния интерфейс организатор не може да промени чужд турнир. Първият фрагмент по-долу е от контролера, а вторият – от услугата:

```
@PutMapping("/{id}")
@PreAuthorize("hasAnyRole('ORGANIZER', 'ADMINISTRATOR')")
public TournamentResponse update(@PathVariable Long id,
    @Valid @RequestBody UpdateTournamentRequest request,
    Authentication authentication) {
    return service.update(id, request, authentication);
}

private void requireOwner(Tournament t, Authentication a) {
    boolean isAdmin = a.getAuthorities().stream()
        .anyMatch(x -> x.getAuthority().equals("ROLE_ADMINISTRATOR"));
    if (!isAdmin && !t.getOrganizer().getUsername().equals(a.getName())) {
        throw new UnauthorizedOperationException(
            "You do not own this tournament");
    }
}
```

Участници се добавят по два начина. Организаторът въвежда име на участник или отбор и по желание го свързва със съществуващ потребителски профил (Фигура 6). Потребител с роля участник може сам да се запише в турнир с отворена регистрация, като посочи име за турнира, например име на отбор, или остави потребителското си

име, и може да се откаже, докато мачовете не са генерирани. И в двата случая една и съща вътрешна операция проверява, че регистрацията е отворена, че името не се повтаря без значение от регистъра на буквите и че един профил участва най-много веднъж в турнира. Нарушенията се връщат с код 409, а проверката за уникалност е подсилена от уникален ключ в базата данни.



Фигура 6. Участници на турнир в регистрация и форма за добавяне

За турнир с групов фаза организаторът създава групи и разпределя всеки участник в точно една от тях (вж. Фигура 22 в Приложение Б). При преместване в друга група участникът се премахва от старата и се добавя в новата в една транзакция, като се обновяват и двете страни на връзката, а повторното разпределение в същата група не променя нищо.

4.4. Модул „Схема“ – генериране на елиминационна схема

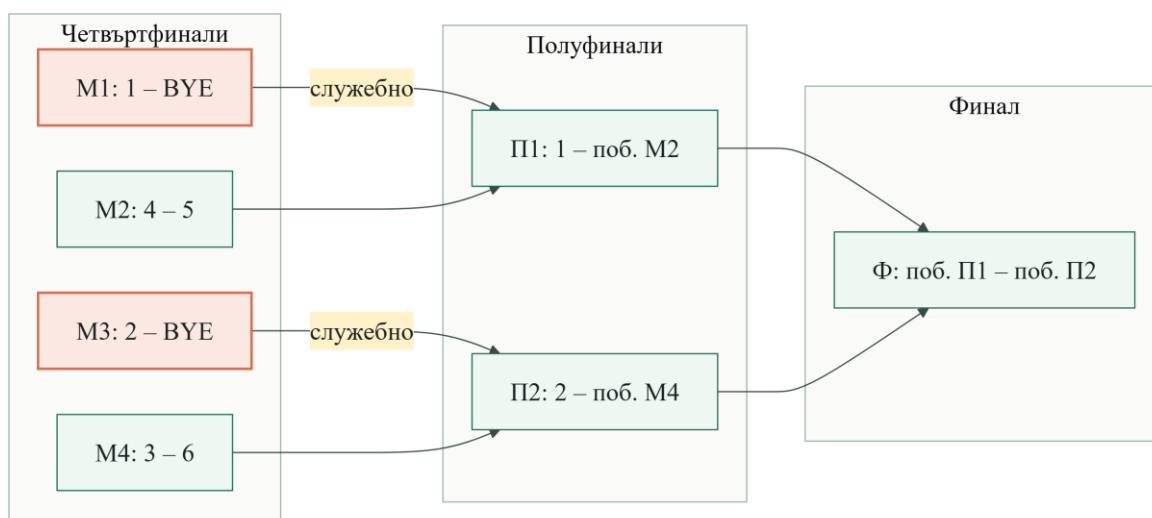
Алгоритъмът е реализиран в компонента BracketGenerator и се състои от три части: изграждане на схемата с поставяне на участниците, уреждане на мачовете (служебни победи и готовност за игра) и придвижване на победителя след всеки въведен резултат.

4.4.1. Размер на схемата и поставяне

Броят на позициите S е най-малката степен на двойката, която не е по-малка от броя на участниците n , а броят на кръговете е $\log_2(S)$. Участниците се поставят по реда на регистрация – първият регистриран е поставен под номер 1. Позицията на всеки поставен номер се получава чрез стандартната подредба, която се строи с удвояване: всеки номер s от схемата с половин размер се следва от своя противник $S + 1 - s$. Така от $[1, 2]$ се получава $[1, 4, 2, 3]$, а след това $[1, 8, 4, 5, 2, 7, 3, 6]$:

```
static int[] seedOrder(int bracketSize) {
    int[] order = {1};
    while (order.length < bracketSize) {
        int size = order.length * 2;
        int[] expanded = new int[size];
        for (int i = 0; i < order.length; i++) {
            expanded[2 * i] = order[i];
            expanded[2 * i + 1] = size + 1 - order[i];
        }
        order = expanded;
    }
    return order;
}
```

Номерата, по-големи от n , съответстват на празни позиции. Празният номер $s > n$ винаги е срещу номер $S + 1 - s$, който не надвишава $S - n$, т.е. срещу един от първите $S - n$ поставени участници. Следователно свободните позиции получават точно най-силните участници, а два празни номера никога не попадат в един мач. Разпределението при 6 участници е показано на Фигура 7.



Фигура 7. Схема за 6 участници – двамата най-високо поставени получават BYE

Всички мачове на схемата се създават наведнъж. Мач с номер m в кръг r се захранва от мачове $2m - 1$ и $2m$ на предходния кръг, а победителят му преминава в мач

$(m + 1)/2$ (с целочислено деление) на следващия кръг – на първа позиция при нечетно m и на втора при четно. Връзката се записва и в полето `nextMatch`, но алгоритъмът работи само с позицията (кръг, номер), затова се държи еднакво с новосъздадени обекти и със заредени от базата данни.

4.4.2. Уреждане на мачовете и свободните позиции

Най-важната част от алгоритъма е методът `propagate`, който обхожда мачовете кръг по кръг и определя състоянието на всяка страна на недовършен мач: **попълнена** – участникът е известен; **празна** – няма захранващ мач (свободна позиция в първия кръг) или захранващият мач е приключил без победител; **изчакваща** – захранващият мач още не е изигран. Мач с две попълнени страни става готов за игра (READY), мач с една попълнена и една празна страна се печели служебно, а мач с поне една изчакваща страна остава в очакване (PENDING):

```
for (TournamentMatch match : ordered) {
    if (match.getStatus() == MatchStatus.COMPLETED) continue;
    Slot first = slot(match.getParticipant1(), feeder(index, match, 1));
    Slot second = slot(match.getParticipant2(), feeder(index, match, 2));
    if (first == Slot.WAITING || second == Slot.WAITING) {
        continue; // a feeder match is still undecided
    }
    if (first == Slot.FILLED && second == Slot.FILLED) {
        if (match.getStatus() != MatchStatus.READY) {
            match.setStatus(MatchStatus.READY);
            becameReady.add(match);
        }
        continue;
    }
    match.setWinner(first == Slot.FILLED // one side is empty: BYE
        ? match.getParticipant1() : match.getParticipant2());
    match.setStatus(MatchStatus.COMPLETED);
    placeWinner(index, match);
}
```

Разграничението между празна и изчакваща страна отстранява дефект от първоначалната версия на проекта. Там всеки мач с липсващ участник се приключваше веднага, затова при 3, 5, 6 или 7 участници участник с BYE ставаше шампион още при генерирането на схемата, без да изиграе нито един мач. Сега служебна победа е възможна само при действително празна позиция, а благодарение на стандартното поставяне това се случва единствено в първия кръг.

Когато организаторът въведе резултат, методът `advance` поставя победителя в следващия мач и отново извиква `propagate`. Методът връща мачовете, които току-що

са станали готови за игра, за да бъдат изпратени известия. Сложността на генерирането и на всяко придвижване е $O(S)$ по време и по памет.

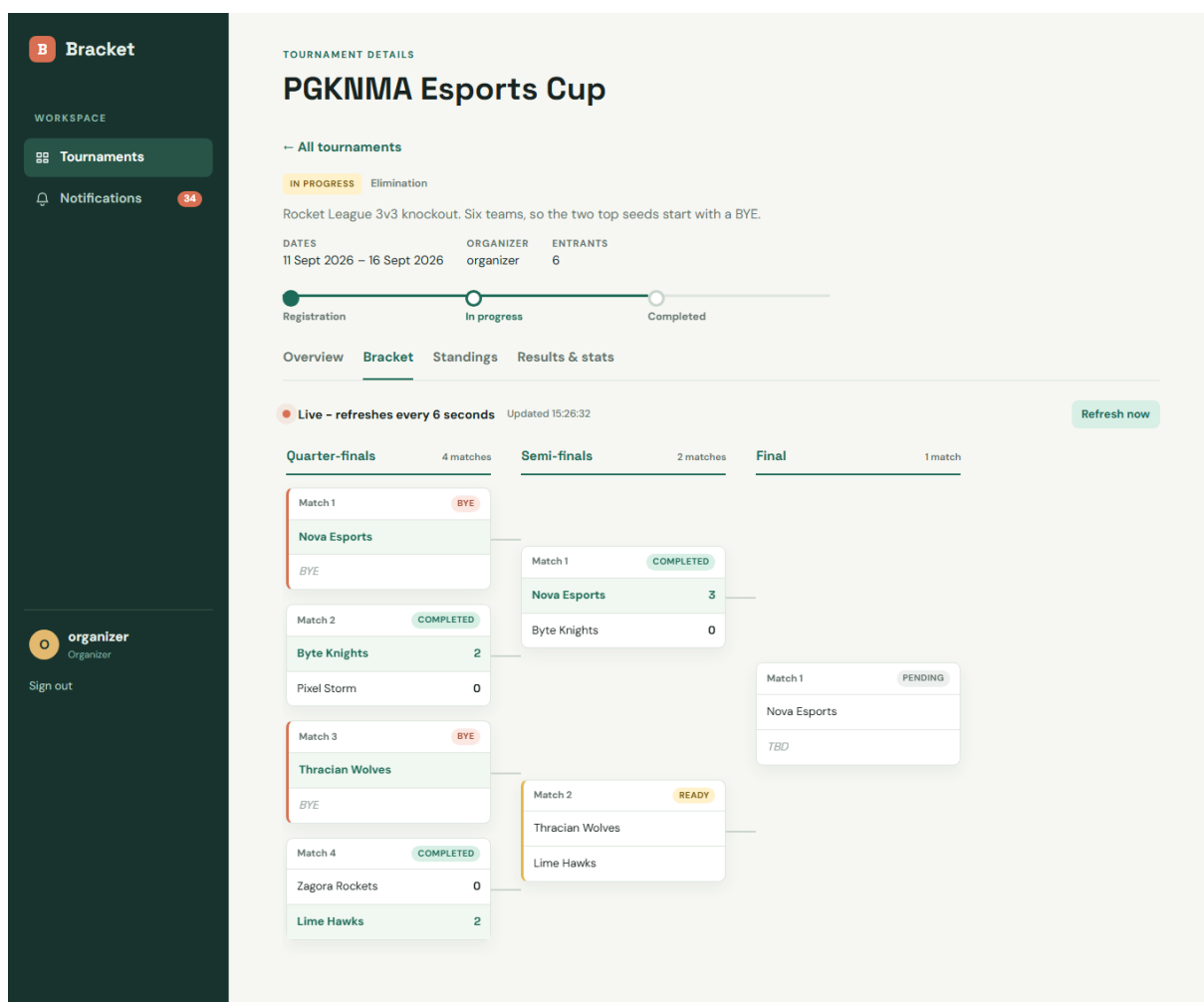
4.4.3. Запис на схемата в базата данни

Всеки ред в таблицата с мачовете сочи към следващия мач чрез външен ключ. Ако мачовете от първия кръг бъдат записани преди финала, Hibernate трябва да запише препратка към обект, който още не съществува в базата данни, и операцията завършва с грешка. Затова услугата записва мачовете в обратен ред – от финала към първия кръг – и едва след това сменя статуса на турнира и изпраща известия за готовите мачове:

```
List<TournamentMatch> generated = switch (tournament.getFormat()) {
    case GROUPS -> groupMatches(tournament, seeding);
    case DOUBLE_ELIMINATION ->
        doubleEliminationGenerator.generate(tournament, seeding);
    case ELIMINATION -> bracketGenerator.generate(tournament, seeding);
};

// later rounds first: every row points at its next-round match
matches.saveAll(generated.stream().sorted(BRACKET_ORDER.reversed()).toList());
tournament.setStatus(TournamentStatus.IN_PROGRESS);
tournaments.save(tournament);
generated.stream()
    .filter(m -> m.getStatus() == MatchStatus.READY)
    .forEach(this::notifyUpcomingMatch);
```

Операцията се изпълнява в една транзакция (@Transactional) – при грешка в базата данни не остава частично записана схема. Схемата на турнир в ход е показана на Фигура 8: двата най-високо поставени отбора са преминали служебно, а финалът изчаква втория полуфинал. Приключилата схема с изведения шампион е дадена на Фигура 23 в Приложение Б. Интерфейсът обновява схемата автоматично на всеки 6 секунди.



Фигура 8. Схема на турнир в ход с BYE за двата най-високо поставени отбора

4.5. Модул „Схема“ – двойна елиминация

При двойната елиминация участникът отпада едва след втората си загуба (Таблица 1). Форматът е реализиран от отделен компонент `DoubleEliminationGenerator`, а не чрез разширяване на съществуващия генератор: схемата на загубилите се захранва по два различни начина, което се описва много по-ясно с явни връзки, отколкото с позиционни формули. Поставянето обаче е общо – новият компонент използва същия метод `seedOrder` от раздел 4.4.1, така че основната схема е идентична на тази при директните елиминации.

За схема с размер $S = 2^k$ структурата се състои от k кръга в основната схема (winners bracket), $2k - 2$ кръга в схемата на загубилите (losers bracket) и голям финал – общо $2S - 2$ мача. Кръговете в схемата на загубилите се редуват: в нечетните се срещат оцелелите в нея, а в четните към тях се присъединяват току-що отпадналите от основната схема, така че никой не среща два пъти подред новоотпаднал противник. Загубилите в първи кръг на основната схема започват от първи кръг на схемата на

загубилите, а загубилите в кръг $r > 1$ влизат в кръг $2(r - 1)$. Кръг j от схемата на загубилите съдържа $2^{(k - 1 - \lfloor j/2 \rfloor)}$ мача.

Таблица 8. Структура на схемата при двойна елиминация с 8 участници

Част	Кръгове	Мачове	Откъде идват участниците
Основна схема	1 – 3	$4 + 2 + 1 = 7$	Поставяне по реда на регистрация
Схема на загубилите	1 – 4	$2 + 2 + 1 + 1 = 6$	Загубилите от основната схема и оцелелите в нея
Голям финал	1	1 (+ 1 реванш)	Победителите в двете схеми

4.5.1. Номериране на кръговете и запис в базата данни

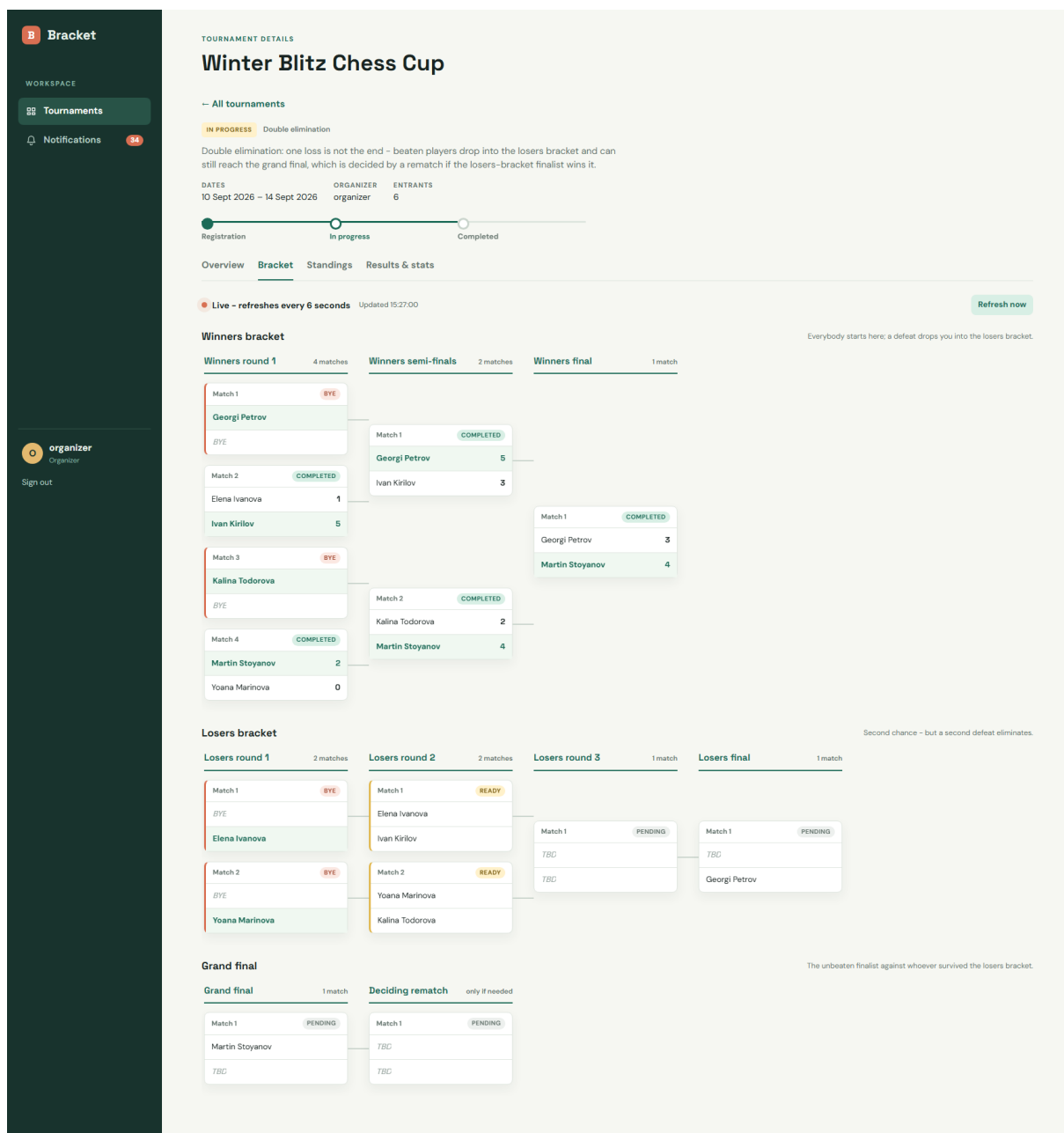
Мачовете се адресират с двойката (кръг, номер), която е уникална за турнира чрез ограничението `uk_match_round_number`. За да остане това ограничение непроменено, кръговете на схемата на загубилите се номерират след тези на основната схема: основна схема $1..k$, схема на загубилите $k+1..3k-2$, голям финал $3k-1$ и евентуален реванш $3k$. Освен че запазва уникалността, това номериране има и второ предимство – всяка връзка води към строго по-голям номер на кръг. Затова уреждането на схемата е едно преминаване във възходящ ред, а записът в базата данни се извършва в низходящ ред на кръговете, при което външните ключове `next_match_id` и `next_loser_match_id` винаги сочат към вече записан ред (същият подход като в раздел 4.4.3).

Всеки мач пази къде отива победителят (`nextMatch` и номер на позиция) и къде отива загубилият (`nextLoserMatch` и номер на позиция). Връзките се разрешават по идентификатор в рамките на вече заредения списък с мачове, а не чрез обръщение към полето на свързания обект – така мързеливият заместител (`проху`) никога не се зарежда и алгоритъмът работи еднакво върху новосъздадена и върху прочетена от базата данни схема.

```
for (int round = 1; round <= winnersRounds; round++) {
    int count = 1 << (winnersRounds - round);
    for (int number = 1; number <= count; number++) {
        TournamentMatch match = index.get(new Position(round, number));
        // Победителят се придвижва както при единичната схема (раздел 4.4);
        // новото е, че загубилият не отпада, а пада в схемата на загубилите:
        Position target = round == 1
            ? new Position(winnersRounds + 1, (number + 1) / 2)
            : new Position(winnersRounds + 2 * (round - 1), number);
        setLoserNext(match, index.get(target), round == 1 ? side(number) : 2);
    }
}
```

4.5.2. Свободни позиции и голям финал

Свободните позиции се обработват по същата тристепенна логика като при директните елиминации, но с едно допълнение: мач, спечелен служебно, няма загубил. Съответната позиция в схемата на загубилите остава празна и противникът ѝ преминава напред служебно – затова при 6 участници първият кръг на схемата на загубилите се състои от два служебни мача (Фигура 9). Позиция, чийто захранващ мач още не е изигран, остава в състояние „изчаква“ и мачът не става готов за резултат.



Фигура 9. Турнир с двойна елиминация: основна схема, схема на загубилите и голям финал

Победителят в основната схема стига до големия финал без загуба, а противникът му има една. Дали това предимство се изразява в правилата, решава

организаторът при създаването на турнира (Фигура 10). Когато реваншът е включен, мачът съществува в схемата от самото начало: ако победи участникът от схемата на загубилите, двамата имат по една загуба и реваншът става готов за игра със същите двама участници; ако победи непобеденият, реваншът се отбелязва като приключил без победител и без участници – „не е необходим“. Така турнирът може да завърши, а схемата запазва следа, че мачът е бил предвиден; той не се брои нито за изигран, нито за служебен.

The screenshot shows the 'New tournament' interface. On the left is a dark sidebar with 'Bracket' logo, 'Tournaments' menu, and 'organizer' profile. The main area is titled 'New tournament' with a 'Back to tournaments' link. The 'Create tournament' section includes a text input for 'Name' (filled with 'Autumn Blitz Cup'), a dropdown for 'Format' (selected 'Double elimination - a loss drops you into a losers bracket'), a checked checkbox for 'Play a deciding rematch when the losers-bracket finalist wins the grand final' with explanatory text, date pickers for 'Start date' and 'End date' (both set to 09/12/2026), and a 'Description' text area with placeholder text 'Rules, venue, prizes...'. A 'Sign out' button is at the bottom of the sidebar.

Фигура 10. Избор на формат и на правилото за решаващ реванш при създаване на турнир

```
private void settleDecider(TournamentMatch decider,
    TournamentMatch grandFinal, List<TournamentMatch> becameReady) {
    if (grandFinal == null
        || grandFinal.getStatus() != MatchStatus.COMPLETED) {
        return;
    }
    if (wonBy(grandFinal, grandFinal.getParticipant2())) {
        decider.setParticipant1(grandFinal.getParticipant1());
        decider.setParticipant2(grandFinal.getParticipant2());
        if (decider.getStatus() != MatchStatus.READY) {
            decider.setStatus(MatchStatus.READY);
            becameReady.add(decider);
        }
        return;
    }
    // не е необходим: непобеденият спечели големия финал
    decider.setStatus(MatchStatus.COMPLETED);
}
```

Класирането използва същия механизъм като при директните елиминации: подреждане по това докъде е стигнал участникът, изразено чрез номера на кръга, в който е загубил. Благодарение на избраното номериране това остава вярно и тук – колкото по-късно отпада един участник от схемата на загубилите, толкова по-напред се класира. Свойството, което се проверява автоматично (раздел 5.2), е, че след изиграването на цялата схема всеки участник без шампиона има точно две загуби, а шампионът – най-много една.

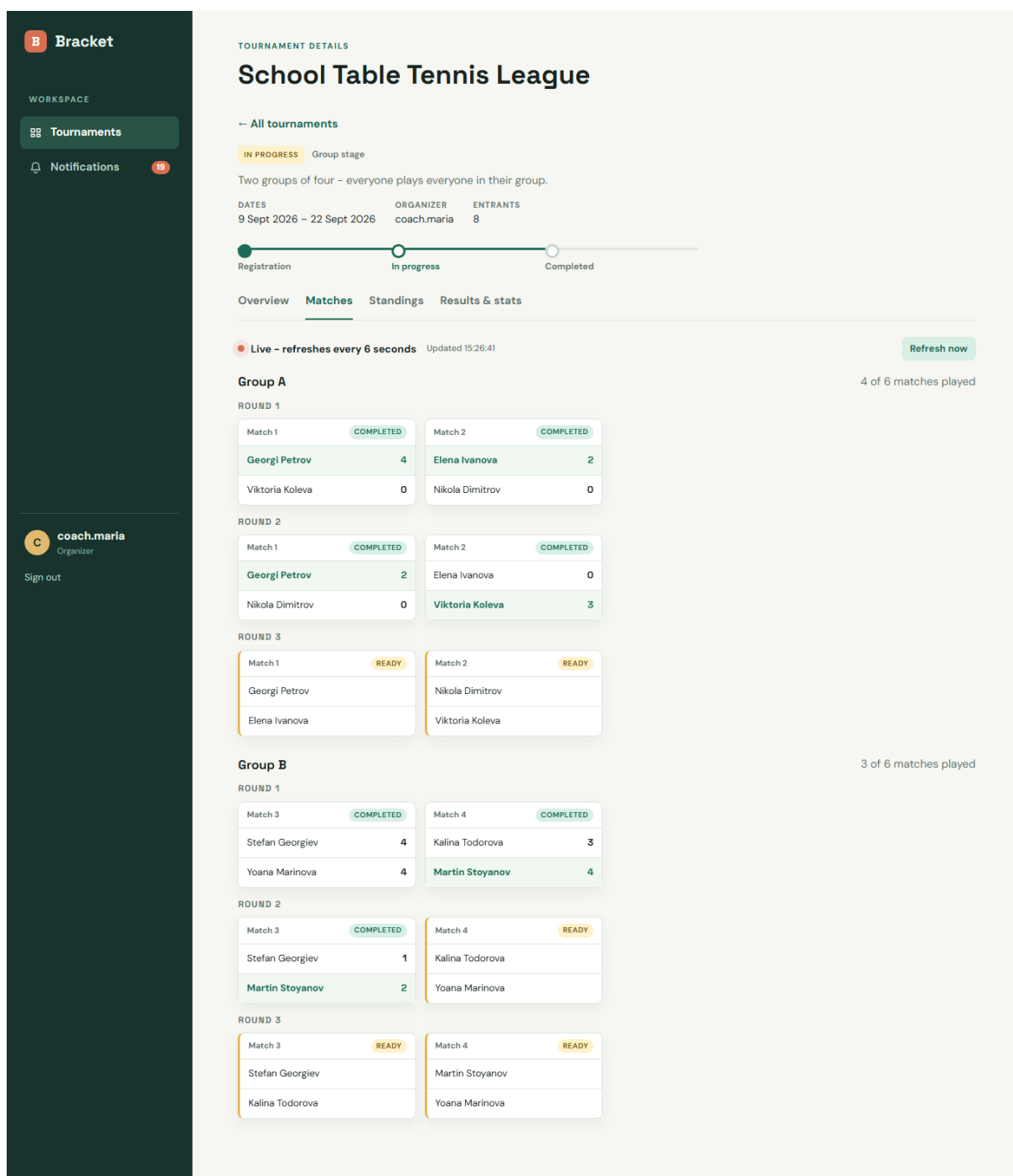
4.6. Групова фаза – график „всеки срещу всеки“

За турнирите с групи мачовете се генерират за всяка група по метода на кръга, на който се основават таблиците на Бергер [1]. Позициите на участниците се подреждат в кръг; във всеки кръг позиция i играе срещу позиция $p - 1 - i$, след което всички позиции без първата се завъртат с една стъпка. При нечетен брой участници се добавя фиктивна позиция „почивка“ и участникът срещу нея не играе в съответния кръг. Резултатът е $p - 1$ кръга (p при нечетен брой), в които всяка двойка се среща точно веднъж и никой не играе два пъти в един кръг:

```
for (int round = 0; round < slots - 1; round++) {
    List<int[]> pairs = new ArrayList<>();
    for (int i = 0; i < slots / 2; i++) {
        int a = ring[i];
        int b = ring[slots - 1 - i];
        if (a < participantCount && b < participantCount) {
            pairs.add(new int[]{Math.min(a, b), Math.max(a, b)});
        }
    }
    rounds.add(pairs);

    int last = ring[slots - 1]; // rotate all but the first
    System.arraycopy(ring, 1, ring, 2, slots - 2);
    ring[1] = last;
}
```

Преди генерирането услугата проверява, че има поне една група, че всички участници са разпределени и че нито една група няма само един участник. Принадлежността към групата се сравнява по идентификатор, а не по обект, защото Hibernate може да върне заместител (проху) на същия запис. Номерата на мачовете са уникални в рамките на кръга за целия турнир, което съответства на уникалния ключ в базата данни. Мачовете на групите, подредени по кръгове, са показани на Фигура 11.

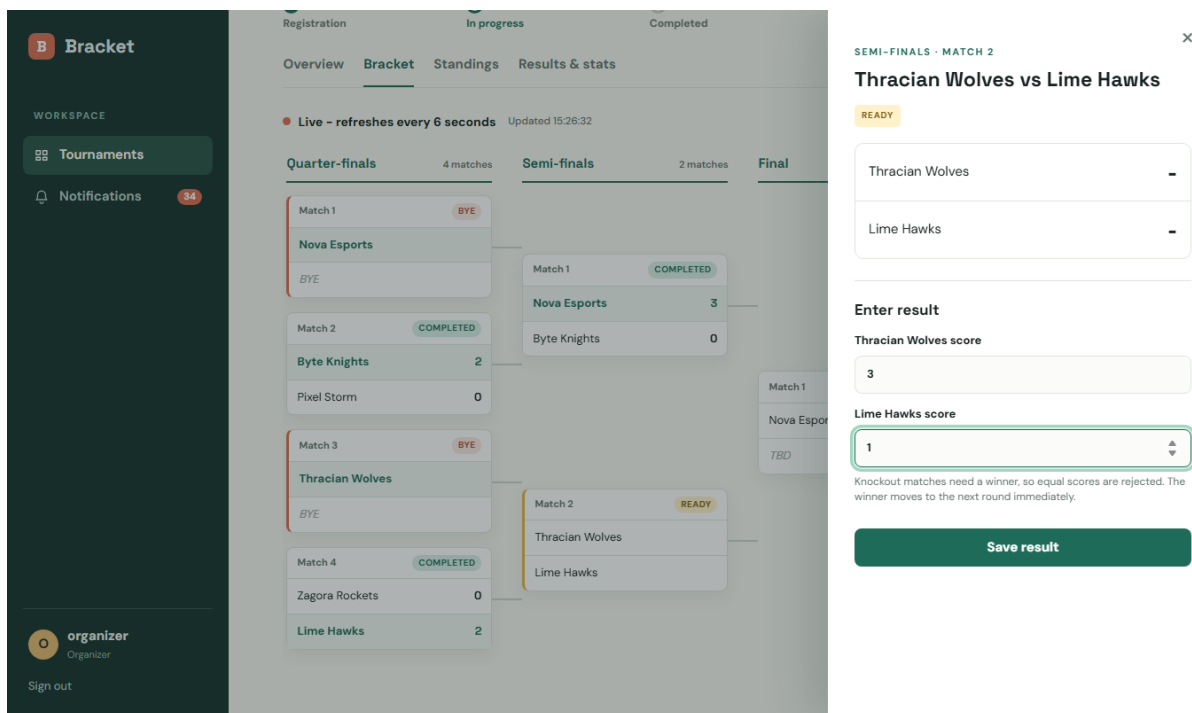


Фигура 11. Мачове от груповата фаза, подредени по групи и кръгове

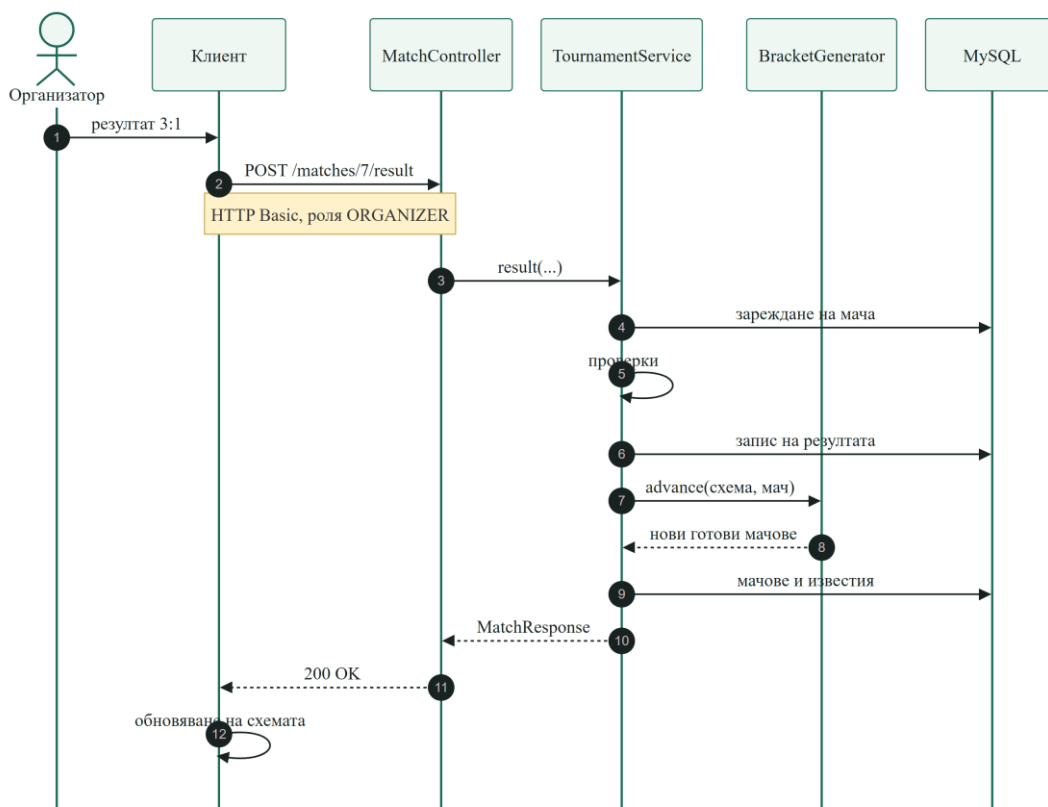
4.7. Модул „Резултати“ – резултати, класиране и статистика

4.7.1. Въвеждане на резултат

Организаторът отваря мач със статус „готов“ и въвежда резултата в страничен панел (Фигура 12). Последователността на обработката на сървъра е показана на Фигура 13.



Фигура 12. Въвеждане на резултат от организатора



Фигура 13. Последователност на обработката при въвеждане на резултат

Услугата проверява, че потребителят е собственик на турнира или администратор, че турнирът е в ход, че мачът е готов за игра, че резултатите не са отрицателни и че при елиминации няма равенство. Едва след всички проверки

резултатът и победителят се записват, победителят се придвижва в схемата и се създават известията:

```
match.setScore1(request.score1());
match.setScore2(request.score2());
Participant winner = isTie ? null : request.score1() > request.score2()
    ? match.getParticipant1() : match.getParticipant2();
match.setWinner(winner);
match.setStatus(MatchStatus.COMPLETED);
matches.save(match);

if (tournament.getFormat() != TournamentFormat.GROUPS) {
    List<TournamentMatch> bracket =
        matches.findByTournamentIdOrderByRoundNumberAscMatchNumberAsc(
            tournament.getId());
    List<TournamentMatch> becameReady =
        tournament.getFormat() == TournamentFormat.DOUBLE_ELIMINATION
            ? doubleEliminationGenerator.advance(bracket, match)
            : bracketGenerator.advance(bracket, match);
    matches.saveAll(bracket);
    becameReady.forEach(this::notifyUpcomingMatch);
}
notifyMatchResult(tournament, match, winner);
publishFinalResultsIfComplete(tournament);
```

Методът е транзакционен, а мачовете и турнирите имат версия за оптимистично заключване. Ако двама организатори запишат резултат за един и същ мач едновременно, успява само първата транзакция, а втората получава отказ – с код 409 при едновременен запис или с код 400, ако мачът вече е приключил. След последния мач статусът на турнира става „Приключил“ и повече резултати не се приемат.

4.7.2. Класиране

Класирането се изчислява от компонента `RankingCalculator` винаги наново от записаните мачове, затова не може да се разминава с тях. За всеки участник се натрупват изиграни мачове, победи, равенства, загуби, отбелязани и допуснати точки, като мачовете, решени служебно, не се броят за изиграни. Точките са 3 за победа и 1 за равенство. При групите подредбата е по победи, разлика в резултата и ред на регистрация. При двата елиминационни формата първият критерий е достигнатият етап – шампионът е над финалиста, финалистът е над отпадналите на полуфинал и т.н.; при двойната елиминация етапът е кръгът, в който участникът е загубил за втори път:

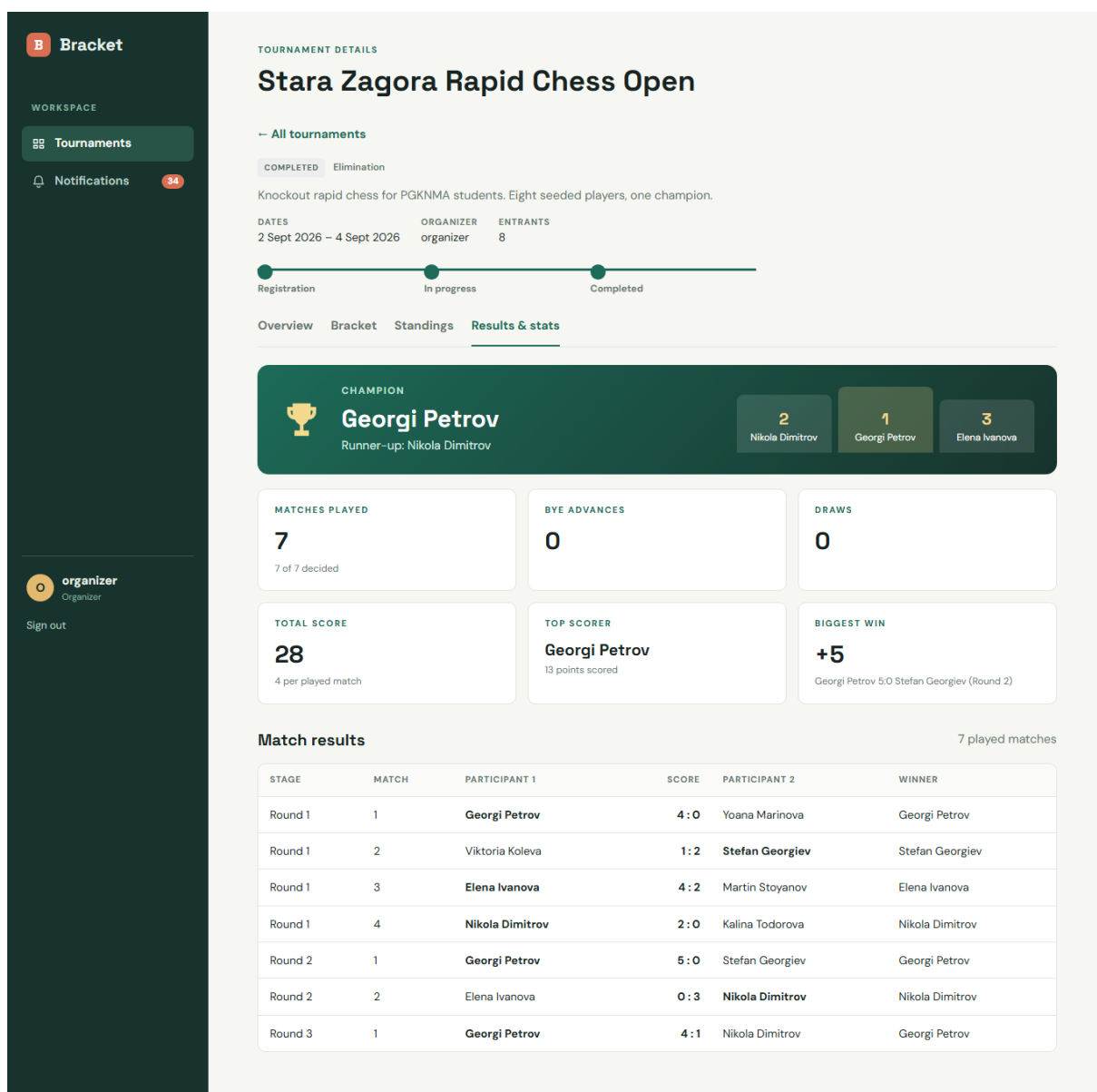
```
List<Long> ordered = stats.keySet().stream()
    .sorted(Comparator
        .comparingInt((Long id) -> isElimination
            ? progression.getOrDefault(id, 1) : 0).reversed())
    .thenComparing(Comparator
        .comparingInt((Long id) -> stats.get(id).wins).reversed())
    .thenComparing(Comparator
```

```
        .comparingInt((Long id) -> scoreDifference(stats.get(id)))  
        .reversed()  
        .thenComparingLong(id -> id))  
    .toList();
```

Последният критерий – идентификаторът на участника – гарантира, че подредбата е детерминирана. Класирането на приключил турнир с директни елиминации е показано на Фигура 24 в Приложение Б.

4.7.3. Статистика и публикуване на крайните резултати

Компонентът `TournamentStatisticsCalculator` изчислява показатели за целия турнир: общ брой мачове, приключени и оставащи мачове, процент на напредък, изиграни мачове, служебни победи, равенства, сума и среден брой точки на мач, участник с най-много отбелязани точки и най-голяма победа. Шампионът и финалистът се публикуват едва когато турнирът е приключил – при елиминации това са победителят и загубилият финала, а при групи – първите двама в класирането. Разделът „Резултати и статистика“ на приключил турнир е показан на Фигура 14.



Фигура 14. Шампион, подиум, статистика и резултати от мачовете

4.8. Модул „Известия“

Известията се създават от услугата в момента на събитието и в същата транзакция, в която то се записва. Реализирани са три вида известия, обобщени в Таблица 9.

Таблица 9. Видове известия

Събитие	Вид	Получатели
Мач става готов за игра – при генериране на схемата или след придвижване на победител	Предстоящ мач	Двамата участници със свързан профил и организаторът
Въведен резултат	Резултат от мач	Организаторът
Приключване на турнира	Крайни резултати	Организаторът и всички участници със свързан профил

Преди записване се проверява дали същият получател вече има известие от същия вид за същия мач или турнир, затова известията не се дублират. Текстът на известието за предстоящ мач съдържа името на турнира, кръга, номера на мача и противника. Участниците без свързан профил не получават известия, тъй като няма към кого да бъдат изпратени.

Клиентът проверява за нови известия на всеки 30 секунди и показва броя на непочетените до елемента от менюто. Проверката се пропуска, докато разделът на браузъра е скрит, а при недостъпен сървър се запазва последната стойност и следващата проверка опитва отново:

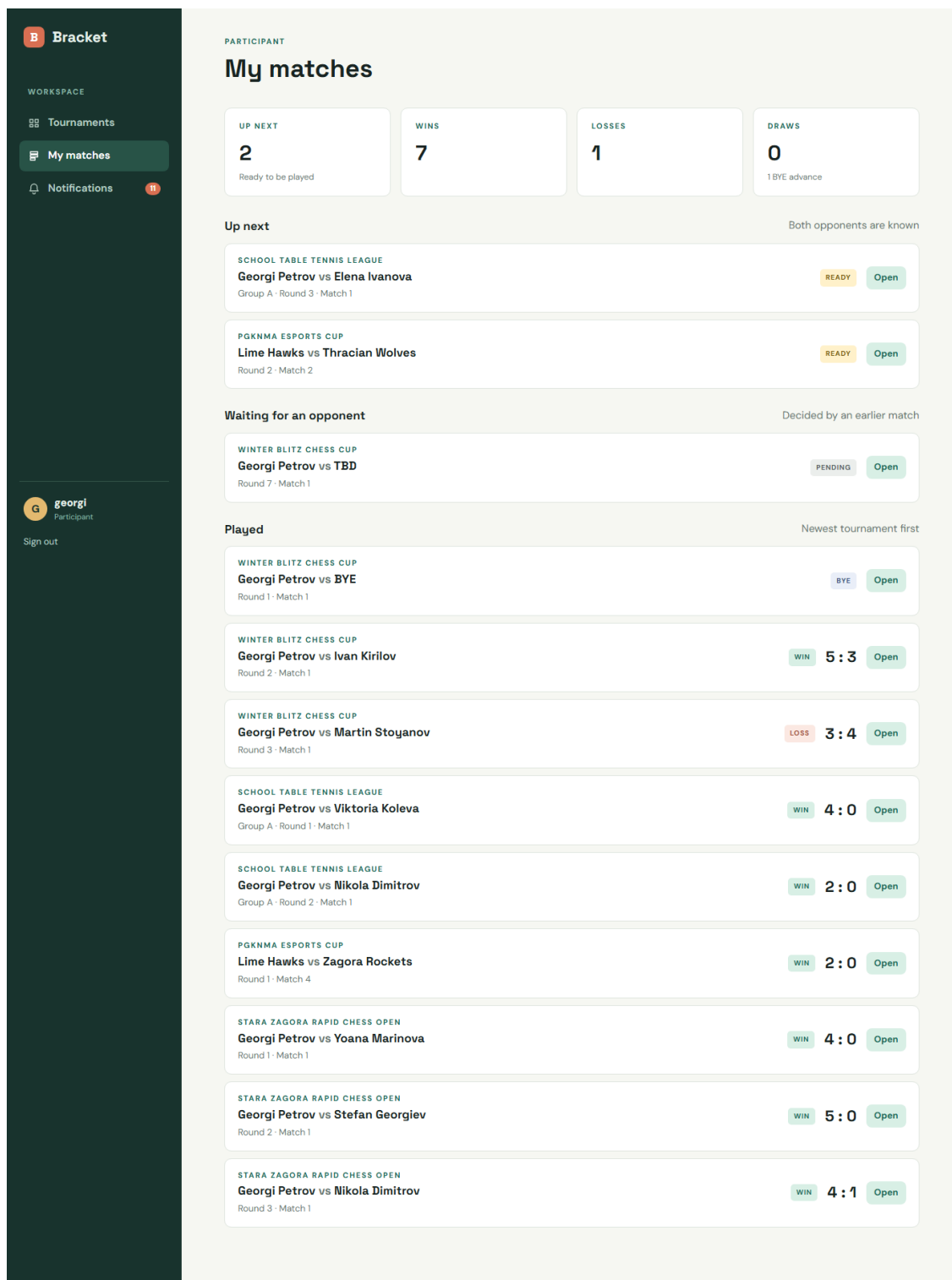
```
function startNotificationPolling() {
  stopNotificationPolling();
  state.notificationPollTimer = setInterval(() => {
    if (!document.hidden) loadNotifications();
  }, NOTIFICATION_POLL_INTERVAL_MS);
}

async function loadNotifications() {
  if (!state.credentials) return;
  try {
    const unread = await request('/notifications?unread=true');
    updateBadge(unread.length);
  } catch {
    // keep the last known badge; the next poll retries
  }
}
```

Списъкът с известия позволява отбелязване на отделно известие или на всички като прочетени и отваряне на съответния турнир (вж. Фигура 26 в Приложение Б).

4.9. Функции за участници и администратори

Страницата „Моите мачове“ (Фигура 15) събира мачовете от всички турнири, в които участва вписаният потребител, разделени на предстоящи, очакващи противник и изиграни. Резултатът се показва от гледна точка на потребителя, а изходът е означен като победа, загуба, равенство или ВУЕ. Данните се извличат с една заявка с граф на обектите, която зарежда турнира, участниците, победителя и групата наведнъж.



Фигура 15. Страница „Моите мачове“ на участник

Администраторският панел (Фигура 16) показва обобщение на платформата – брой профили по роли, блокирани профили, турнири по статуси и изиграни мачове – и списък с потребителите. Администраторът променя ролята на потребител и блокира или възстановява достъпа му, като всяко действие се потвърждава в диалогов прозорец. Администраторът не може да промени собствената си роля или да блокира себе си, а

последният администратор не може да бъде понижен – така платформата никога не остава без администратор. Тъй като автентикацията не използва сесия, промяната важи от следващата заявка на засегнатия потребител, а блокиран потребител получава отказ още при опит за вход.

Bracket

WORKSPACE

- Tournaments
- Notifications
- Administration**

admin
Administrator
Sign out

Administration

ACCOUNTS
15
1 admin · 2 organizers · 12 participants

BLOCKED ACCOUNTS
0

TOURNAMENTS
5
1 open · 3 live · 1 completed

MATCHES PLAYED
22

User accounts Changes apply on the user's next request.

USER	EMAIL	ROLE	STATUS	JOINED	
A admin YOU	admin@example.com	Administrator	ACTIVE	12 Sept 2026	
O organizer	organizer@example.com	Organizer	ACTIVE	12 Sept 2026	Block
C coach.maria	coach.maria@example.com	Organizer	ACTIVE	12 Sept 2026	Block
G georgi	georgi@example.com	Participant	ACTIVE	12 Sept 2026	Block
E elena	elena@example.com	Participant	ACTIVE	12 Sept 2026	Block
N nikola	nikola@example.com	Participant	ACTIVE	12 Sept 2026	Block
V viktorija	viktorija@example.com	Participant	ACTIVE	12 Sept 2026	Block
S stefan	stefan@example.com	Participant	ACTIVE	12 Sept 2026	Block
K kalina	kalina@example.com	Participant	ACTIVE	12 Sept 2026	Block
M martin	martin@example.com	Participant	ACTIVE	12 Sept 2026	Block
Y yoana	yoana@example.com	Participant	ACTIVE	12 Sept 2026	Block
P petar	petar@example.com	Participant	ACTIVE	12 Sept 2026	Block
D desislava	desislava@example.com	Participant	ACTIVE	12 Sept 2026	Block
I ivan	ivan@example.com	Participant	ACTIVE	12 Sept 2026	Block
R radostina	radostina@example.com	Participant	ACTIVE	12 Sept 2026	Block

Фигура 16. Администраторски панел за управление на потребителите

4.10. Реализация на нефункционалните изисквания

4.10.1. Сигурност и ролеви достъп

Конфигурацията на Spring Security задава приложение без сесии, при което всяка заявка носи данните за вход. Публични са само адресите за регистрация и вход, адресите на администрацията изискват роля администратор още на ниво филтър, а всички останали изискват успешна автентикация. Грешките при автентикация и

отказан достъп се връщат в същия JSON формат като останалите грешки (във фрагмента обработчиците са съкратени):

```
return http
    .csrf(csrf -> csrf.disable())
    .cors(cors -> {})
    .sessionManagement(session ->
        session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
    .exceptionHandling(exceptions -> exceptions
        .authenticationEntryPoint(/* JSON error with status 401 */)
        .accessDeniedHandler(/* JSON error with status 403 */))
    .authorizeHttpRequests(authorize -> authorize
        .requestMatchers("/api/auth/**", "/error").permitAll()
        .requestMatchers("/api/admin/**").hasRole("ADMINISTRATOR")
        .anyRequest().authenticated())
    .httpBasic(basic -> {})
    .build();
```

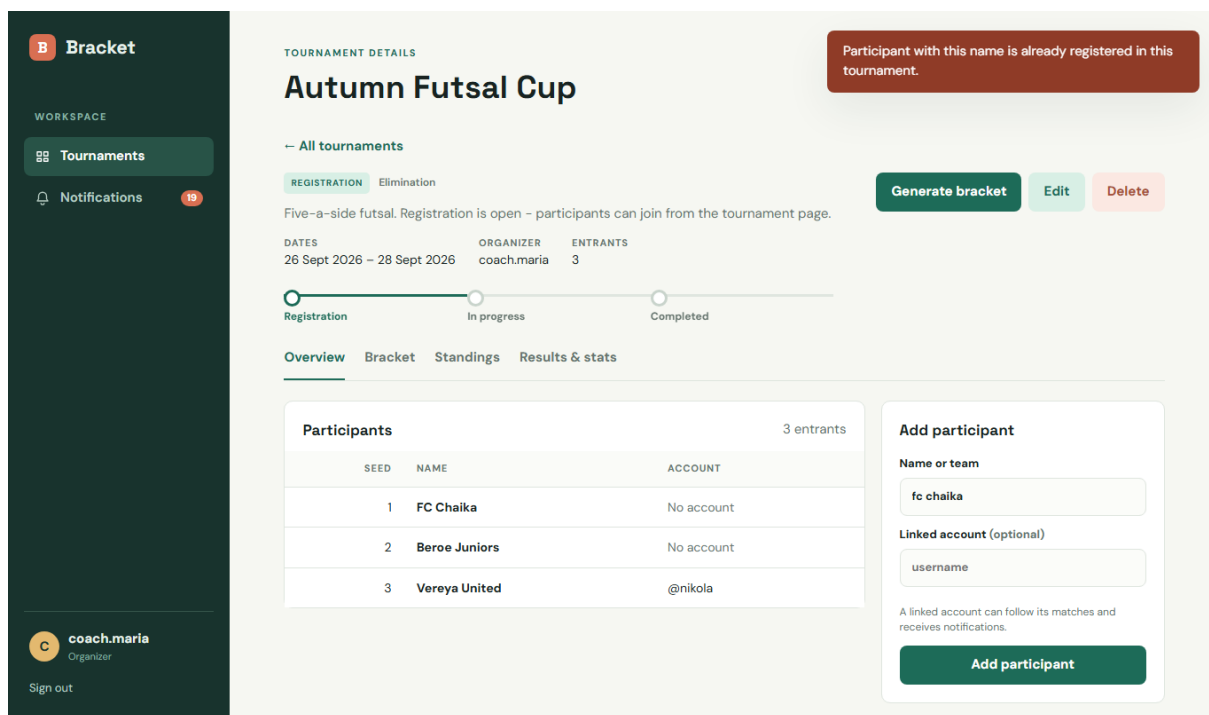
Защитата от CSRF е изключена, защото приложението не използва бисквитки за автентикация и браузърът не изпраща данните за вход автоматично при заявки от чужди сайтове. Паролите се хешират с BCrypt, профилите се зареждат заедно с ролята и флага за активност, а обектите за отговор никога не съдържат хеша на паролата. Проверката на ролите по адрес и чрез @PreAuthorize е допълнена от проверките за собственост в услугите, описани в раздел 4.3. Така са адресирани рисковете „нарушен контрол на достъпа“ и „грешки при идентификация и автентикация“ от списъка OWASP Top 10 [26].

4.10.2. Валидиране и обработка на грешки

Всяко тяло на заявка е описано с Java запис с анотации за валидиране. Контролерът приема обекта с анотация @Valid и при нарушено ограничение заявката изобщо не достига до услугата:

```
public record CreateTournamentRequest(
    @NotBlank(message = "Tournament name is required")
    @Size(max = 150,
        message = "Tournament name must be at most 150 characters")
    String name,
    @Size(max = 2000, message = "Description must be at most 2000
characters")
    String description,
    @NotNull(message = "Tournament format is required")
    TournamentFormat format,
    @NotNull(message = "Start date is required")
    @FutureOrPresent(message = "Start date cannot be in the past")
    LocalDate startDate,
    @NotNull(message = "End date is required")
    LocalDate endDate) { }
```

Глобалният обработчик на изключения (@RestControllerAdvice) превръща всяко изключение в отговор с подходящ код: бизнес грешките – в 400, отказаният достъп – в 403, несъществуващите ресурси – в 404, конфликтите и едновременните промени – в 409, а всички неочаквани грешки – в 500 с общо съобщение без технически подробности. Интерфейсът показва съобщението на сървъра в известие в горния десен ъгъл (Фигура 17).



Фигура 17. Съобщение при опит за дублиране на участник

4.10.3. Цялостност на данните и производителност

Цялостността на резултатите е осигурена на няколко нива, всяко от които спира грешка, пропусната от предходното: валидиране на входа, бизнес правила за статуса на турнира и на мача, транзакции, оптимистично заключване и ограничения в базата данни – уникални ключове, външни ключове и проверки CHECK за неотрицателни резултати и за периода на турнира.

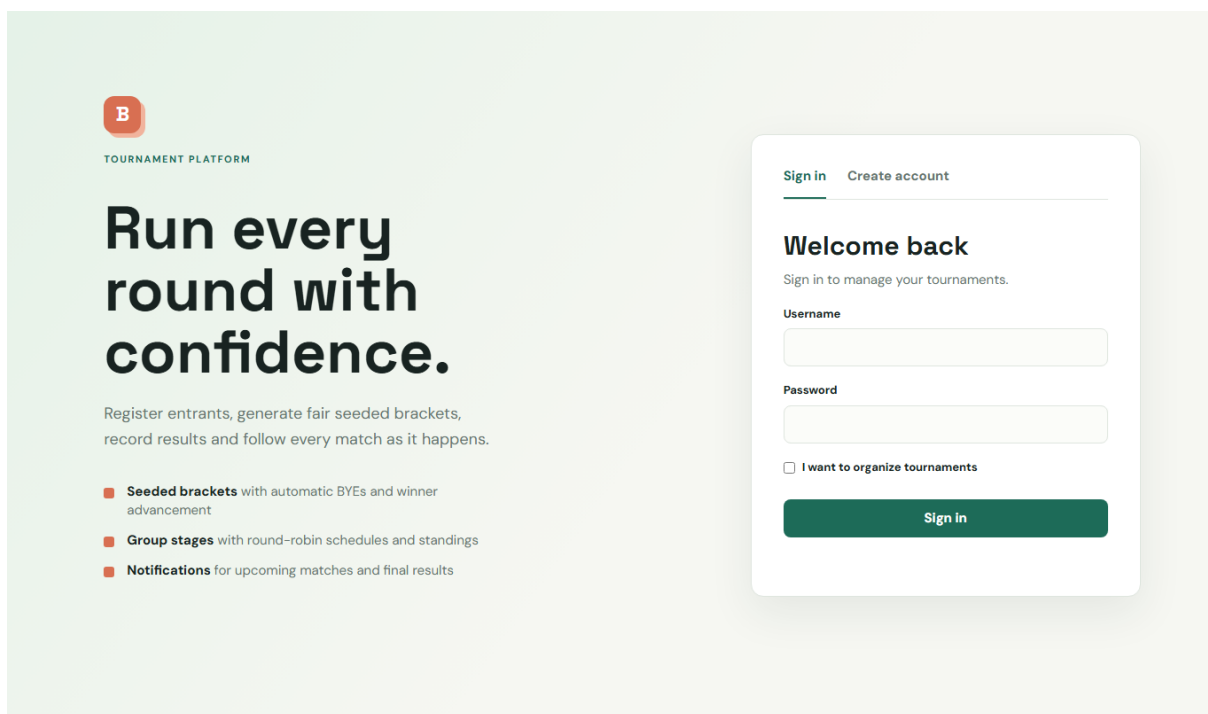
За производителността са премахнати излишните заявки към базата данни. Списъкът с турнири се извлича с една JPQL заявка, която изчислява броя на участниците с COUNT в базата данни, вместо да зарежда всички участници в паметта. Схемата, известията и „Моите мачове“ използват графи на обектите, които зареждат свързаните участници с една заявка вместо с отделна заявка за всеки мач (т.нар. проблем N+1). Добавени са индекси по организатор, начална дата, участници в мачовете и по комбинацията получател – прочитане – дата при известията.

Регистрирането на SQL заявки е изключено по подразбиране, за да не се записват в журнала данни като хешове на пароли.

4.11. Потребителски интерфейс

Уеб клиентът е приложение с една страница, реализирано с HTML, CSS и JavaScript без външни библиотеки. Интерфейсът се адаптира към ролята: организаторът вижда бутоните за създаване и управление, участникът – формата за записване и страницата „Моите мачове“, а администраторът – панела за администрация. Тези разлики улесняват работата, но не са защита – всяко действие се проверява отново от сървъра.

Входът и регистрацията са на обща страница (Фигура 18). Данните за вход се пазят само в паметта на страницата и не се записват в браузъра, затова презареждането изисква повторен вход. Необратимите действия – генериране на схема, изтриване на турнир, блокиране на профил – се потвърждават в диалогов прозорец. Всеки турнир показва етапа от жизнения си цикъл и е разделен на раздели „Преглед“, „Схема“ (или „Мачове“ при групи), „Класиране“ и „Резултати и статистика“. Оформлението използва CSS Grid и Flexbox и се пренарежда за таблет и телефон (вж. Фигура 31 в Приложение Б).



Фигура 18. Страница за вход и регистрация

5. Тестване, резултати и демонстрация

5.1. Подход към тестването

Тестването следва принципа на тестовата пирамида [27]: най-много са бързите модулни тестове на алгоритмите и бизнес правилата, по-малко са тестовете на уеб слой и сигурността, а най-малко са бавните сквозни проверки на работещото приложение с реална база данни и истински браузър. Нивата и използваните средства са обобщени в Таблица 10.

Таблица 10. Нива на тестване

Ниво	Средства	Какво се проверява
Модулни тестове на алгоритмите	JUnit 5, параметризирани тестове	Поставяне, BYE, придвижване, график „всеки срещу всеки“, класиране и статистика – без Spring и без база данни
Тестове на услугите	JUnit 5, Mockito	Бизнес правила: жизнен цикъл, собственост, регистрация, групи, резултати, известия, администрация
Тестове на уеб слой и сигурността	Spring MockMvc, Spring Security Test	Права по роли, кодове 400, 401, 403, 404 и 409, формат на грешките, валидиране
Интеграционни тестове (по избор)	Spring Boot Test с MySQL	Уникални ключове, каскадно изтриване, едновременни промени – при зададена тестова база данни
Сквозни тестове на програмния интерфейс	Node.js скрипт с Fetch API	Работещото приложение с MySQL – от вход до приключване на турнир
Тестове в браузър	Playwright, Chromium	Интерфейсът на известията и заснемане на всички екрани

Особено внимание е отделено на коректността на алгоритъма за схемата. Освен тестове с конкретни очаквани стойности са използвани и тестове с генерирани сценарии: за всеки брой участници от 2 до 64 се симулира цял турнир, в който победителите се избират случайно, а резултатите пристигат в случаен ред. След всяка симулация се проверяват свойства, които са верни за всяка правилна схема – изиграни са точно $n - 1$ мача, не е останал мач в очакване, шампионът не е губил, всеки друг участник е отпаднал точно веднъж, а шампионът е изиграл всички кръгове или всички без един, ако е получил BYE.

5.2. Резултати от автоматизираните тестове

Пълният набор от тестове се изпълнява с командата `mvnw test`. При последното изпълнение с JDK 17.0.20 са изпълнени **212** теста – **208** успешни, **0** неуспешни и **4** пропуснати. Пропуснатите тестове са интеграционните тестове с реална MySQL база

данни, които по замисъл се изпълняват само при зададени променливи за тестова база данни. Разпределението по тестови класове е дадено в Таблица 11.

Таблица 11. Автоматизирани тестове по класове

Тестов клас	Обхват	Брой
BracketGeneratorTest	Подредба на поставянето, ВУЕ при 2–32 участници, придвижване, $n - 1$ мача	27
BracketLifecycleSimulationTest	Свойства на новата схема и случайни пълни турнири с 2–64 участници	32
DoubleEliminationGeneratorTest	Структура и връзки на схемата на загубилите, служебни победи, изиграван и пропуснат реванш, случайни пълни турнири с 2–16 участници	34
TournamentServiceTest	Жизнен цикъл, собственост, регистрация, записване, групи, резултати	33
RankingCalculatorTest, RankingTest	Класиране, равенства, ВУЕ, достигнат етап	13
SecurityAuthorizationTest	Кодове 400, 401, 403, 404 и 409, записване според ролята	11
RoundRobinSchedulerTest	Всяка двойка веднъж, никой два пъти в кръг	10
NotificationFlowTest	Известия за мачове, резултати и крайни резултати	7
AdminSecurityTest, UserAdminServiceTest	Достъп до администрацията, смяна на роли, блокиране	9
Други тестове	Статистика, първи администратор, проверка на схемата на базата данни, валидиране, автентикация, обработка на грешки	32
Интеграционни тестове с MySQL	Ограничения, каскади, едновременни промени	4 (пропуснати)

5.3. Примерни тестови случаи

В Таблица 12 са представени примерни тестови случаи с очаквания и получения резултат. Всички те са част от автоматизираните тестове или от сквозната проверка, описана в следващия раздел.

Таблица 12. Примерни тестови случаи

№	Сценарий	Входни данни	Очакван резултат	Получен резултат
T01	Стандартна подредба на поставянето	Схема с 8 позиции	1, 8, 4, 5, 2, 7, 3, 6	Съвпада
T02	ВУЕ не печели финала	3 участници	Поставеният под № 1 получава ВУЕ; финалът изчаква полуфинала	Съвпада
T03	ВУЕ само за най-високо поставените	2–32 участници	$S - n$ служебни победи, всички в първия кръг	Съвпада
T04	Пълен турнир в случаен ред	2–64 участници	$n - 1$ мача, един шампион, никой не отпада два пъти	Съвпада
T05	График „всеки срещу	2–10 участници	Всяка двойка веднъж,	Съвпада

№	Сценарий	Входни данни	Очакван резултат	Получен резултат
	всеки“		никой два пъти в кръг	
T06	Структура при двойна елиминация	8 участници	14 мача: 7 в основната схема, 6 в схемата на загубилите, 1 голям финал	Съвпада
T07	Отпадане едва след втората загуба	2–16 участници, случаен ред	Всеки освен шампиона има точно две загуби	Съвпада
T08	Реванш само при нужда	Голям финал с включен реванш	При победа на непобедения – без реванш; иначе – реванш със същите двама	Съвпада
T09	Финалът се отключва след двата полуфинала	4 участници, резултати един след друг	В очакване → готов → турнирът приключва	Съвпада
T10	Равен резултат при елиминации	Резултат 2:2	Отказ с код 400	Код 400
T11	Резултат от чужд организатор	Друг организатор	Отказ с код 403, нищо не се записва	Код 403
T12	Дублирано име на участник	„alpha“ при съществуващ „Alpha“	Конфликт с код 409	Код 409
T13	Повторно записване в турнир	Същият участник	Конфликт с код 409	Код 409
T14	Участник създава турнир	POST /api/tournaments	Отказ с код 403	Код 403
T15	Заявка без автентикация	GET /api/tournaments	Отказ с код 401	Код 401
T16	Вход с блокиран профил	Блокиран потребител	Отказ с код 403	Код 403
T17	Администратор променя своята роля	Собствен профил	Отказ с код 400	Код 400

5.4. Сквозна проверка на работещото приложение

Автоматизираните тестове на Maven не използват реална база данни, затова приложението е проверено и в работещ вид. Изграденият JAR файл е стартиран срещу MySQL 8.0.46 с празна база данни и включени демонстрационни данни – пет турнира в различни етапи и 15 профила. Скриптът `e2e/api-smoke.js` изпраща реални HTTP заявки и проверява 52 твърдения: вход и отказ при грешна парола; записана в базата данни схема за 6 отбора с BYE за първите двама поставени и финал, който изчаква полуфиналите; съответствие между шампиона, финала и класирането; график на груповата фаза; записване и отказ от турнир; правилата за достъп; блокиране и възстановяване на профил; пълен турнир с 5 участници, въведени в случаен ред резултати и приключване след точно 4 мача; както и пълен турнир с двойна

елиминация, след който всеки участник без шампиона има точно две загуби. **Всички 52 проверки са успешни.**

Потребителският интерфейс е проверен със скрипта `e2e/verify.js` в браузъра Chromium – успешни са 13 от 13 проверки за известията, включително поведението при недостъпен сървър. Скриптът `e2e/screenshots.js` преминава през всички екрани за четирите вида потребители и за телефонен размер на екрана; снимките на екрана в настоящата разработка са получени с него.

5.5. Открити и отстранени дефекти

Тестването с реални данни и с генерирани сценарии разкри дефекти в първоначалната версия на проекта, които не бяха засечени от съществуващите тестове:

- **Преждевременен шампион.** Мачовете с липсващ участник се приключваха веднага, дори когато участникът още се определяше в предходен мач. При 3, 5, 6 или 7 участници шампионът се определяше при самото генериране на схемата, а съществуващ тест дори очакваше това поведение. Дефектът е отстранен чрез разграничаване на празна и изчакваща позиция (раздел 4.4.2) и чрез стандартно поставяне.
- **Схемата не можеше да бъде записана.** Записваха се само готовите мачове, които сочат към незаписани мачове от следващия кръг, и базата данни отказваше операцията. Проблемът оставаше скрит при тестовете със заместени хранилища. Сега мачовете се записват от финала към първия кръг (раздел 4.4.3).
- **Несинхронизирани групи.** При разпределение в група се променяше само едната страна на връзката и участниците не се разпознаваха като членове на групата, а сравнението по обект не работеше със заместителите на Hibernate. Двете страни на връзката вече се обновяват заедно, а сравнението е по идентификатор.
- **Остарели тестове и документация.** Тестове зависеха от несъществуващи методи или от реална база данни без проверка за наличието ѝ, а документацията описваше груповата фаза като нереализирана. Тестовете са коригирани, а документацията е обновена.

Поуката е, че успешните тестове със заместени зависимости не доказват работата на цялата система – необходими са и проверки на свойства с много сценарии, и сквозни проверки с реална база данни.

5.6. Известни ограничения

- Мачовете нямат зададени дата и час – известието за предстоящ мач съобщава двойката, но не и часа на провеждане.
- Поставянето следва реда на регистрация; не са реализирани ръчно поставяне, поставяне по рейтинг и мач за трето място.
- Поддържат се директни елиминации, двойна елиминация и групова фаза; швейцарската система и елиминационната фаза след групите не са реализирани.
- Резултатите на живо и известията се обновяват чрез периодични заявки, а не чрез изпращане от сървъра; няма известия по имейл.
- Автентикацията HTTP Basic изисква HTTPS в реална среда, а презареждането на страницата изисква повторен вход.
- Интерфейсът е на английски език, а тестове за натоварване не са провеждани.

6. Заключение

6.1. Постигнати резултати

В резултат на разработката е създадена работеща enterprise платформа за организиране на турнири, която изпълнява изискванията на заданието. Поставените в увода задачи са изпълнени, както следва:

- 1) Проучени са форматите на турнирите, правилата за поставяне и класиране и четири съществуващи платформи, а въз основа на това са формулирани функционалните и нефункционалните изисквания (глави 2 и 3).
- 2) Проектирани са клиент-сървър архитектура със слоест сървър, модел на данните от шест таблици, REST интерфейс и ролеви модел с три роли (глава 3).
- 3) Реализиран е алгоритъм за елиминационна схема със стандартно поставяне, правилно разпределение на свободните позиции и автоматично придвижване на победителите, схема с двойна елиминация със схема на загубилите и голям финал, както и график „всеки срещу всеки“ по метода на кръга (раздели 4.4 – 4.6).
- 4) Реализирани са въвеждането на резултати, класирането, статистиката, публикуването на крайните резултати и автоматичните известия (раздели 4.7 и 4.8).
- 5) Осигурени са сигурност, ролеви достъп и многостепенна защита на цялостта на данните и е изграден адаптивен уеб интерфейс за трите роли (раздели 4.9–4.11).
- 6) Системата е проверена със 169 автоматизирани теста, 41 сквозни проверки срещу реална MySQL база данни и проверки в браузър (глава 5).

6.2. Лични изводи от разработката

Разработката показва, че най-трудната част на подобна система не е изграждането на екраните, а коректността на правилата в граничните случаи. Алгоритъмът за схемата изглежда прост при 8 участници, но едва симулирането на много турнири с различен брой участници и различен ред на резултатите разкри, че една свободна позиция може да излъчи шампион без изигран мач. Това утвърди разбирането, че тестовете трябва да проверяват свойства, а не само конкретни примери, и че заместените зависимости скриват проблеми, които се проявяват едва с реална база данни.

Работата със Spring Boot изясни ползата от разделянето на отговорностите: благодарение на отделените алгоритми и услуги дефектите бяха локализирани и отстранени, без да се променят контролерите и клиентът. Практически бяха усвоени транзакциите и оптимистичното заключване, декларативната сигурност със Spring Security и проектирането на REST интерфейс с последователни кодове на състояние. Не на последно място – документирането на решенията и на известните ограничения е толкова важно, колкото и самият код, защото позволява системата да бъде разбрана и развивана от други.

6.3. Насоки за бъдещо развитие

- Планиране на мачовете по дата, час и място и напомняне преди мача.
- Ръчно поставяне и поставяне по рейтинг, мач за трето място, швейцарска система и елиминационна фаза след групите.
- Резултати в реално време чрез WebSocket или Server-Sent Events и известия по имейл.
- Автентикация с токени (JWT или OAuth 2.0) и възстановяване на забравена парола.
- Въвеждане на резултати от самите участници с потвърждение от организатора.
- Интерфейс на български език, публикуване в облачна среда, контрол на версиите с Git и автоматизирано изграждане и тестване при всяка промяна.

Използвана литература

- [1] FIDE. FIDE Handbook, C.05 General Rules and Technical Recommendations for Tournaments, Annex 1: Berger Tables for Round-Robin Tournaments.
<https://handbook.fide.com/> (последен достъп: 11.09.2026 г.)
- [2] Schwenk, A. J. What Is the Correct Way to Seed a Knockout Tournament? The American Mathematical Monthly, vol. 107, no. 2, 2000, pp. 140–150.
- [3] Challonge. Challonge – Bracket Generator and Tournament Manager.
<https://challonge.com/> (последен достъп: 11.09.2026 г.)
- [4] start.gg. start.gg – Tournament Platform for Gaming Communities.
<https://www.start.gg/> (последен достъп: 11.09.2026 г.)
- [5] Toornament. Toornament – Tournament Management Platform.
<https://www.toornament.com/> (последен достъп: 11.09.2026 г.)
- [6] Battlefy. Battlefy – Esports Tournament Platform. <https://battlefy.com/> (последен достъп: 11.09.2026 г.)
- [7] Oracle. Java Platform, Standard Edition 17 Documentation.
<https://docs.oracle.com/en/java/javase/17/> (последен достъп: 11.09.2026 г.)
- [8] Broadcom Inc. (Spring). Spring Boot Reference Documentation, version 3.4.
<https://docs.spring.io/spring-boot/> (последен достъп: 11.09.2026 г.)
- [9] Broadcom Inc. (Spring). Spring Framework Reference Documentation – Web on Servlet Stack. <https://docs.spring.io/spring-framework/reference/web/webmvc.html> (последен достъп: 11.09.2026 г.)
- [10] Eclipse Foundation. Jakarta Persistence 3.1 Specification.
<https://jakarta.ee/specifications/persistence/3.1/> (последен достъп: 11.09.2026 г.)
- [11] Red Hat, Inc. Hibernate ORM 6.6 User Guide.
https://docs.jboss.org/hibernate/orm/6.6/userguide/html_single/Hibernate_User_Guide.html (последен достъп: 11.09.2026 г.)
- [12] Broadcom Inc. (Spring). Spring Data JPA Reference Documentation.
<https://docs.spring.io/spring-data/jpa/reference/> (последен достъп: 11.09.2026 г.)
- [13] Oracle Corporation. MySQL 8.0 Reference Manual.
<https://dev.mysql.com/doc/refman/8.0/en/> (последен достъп: 11.09.2026 г.)

- [14] Broadcom Inc. (Spring). Spring Security Reference Documentation.
<https://docs.spring.io/spring-security/reference/> (последен достъп: 11.09.2026 г.)
- [15] Provos, N., Mazières, D. A Future-Adaptable Password Scheme. Proceedings of the 1999 USENIX Annual Technical Conference, 1999.
<https://www.usenix.org/legacy/events/usenix99/provos/provos.pdf> (последен достъп: 11.09.2026 г.)
- [16] Reschke, J. RFC 7617: The 'Basic' HTTP Authentication Scheme. Internet Engineering Task Force, 2015. <https://www.rfc-editor.org/rfc/rfc7617> (последен достъп: 11.09.2026 г.)
- [17] Eclipse Foundation. Jakarta Bean Validation 3.0 Specification.
<https://jakarta.ee/specifications/bean-validation/3.0/> (последен достъп: 11.09.2026 г.)
- [18] The Project Lombok Authors. Project Lombok. <https://projectlombok.org/> (последен достъп: 11.09.2026 г.)
- [19] The Apache Software Foundation. Apache Maven – Documentation.
<https://maven.apache.org/guides/> (последен достъп: 11.09.2026 г.)
- [20] Mozilla. MDN Web Docs – Fetch API. https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API (последен достъп: 11.09.2026 г.)
- [21] The JUnit Team. JUnit 5 User Guide. <https://junit.org/junit5/docs/current/user-guide/> (последен достъп: 11.09.2026 г.)
- [22] Mockito. Mockito Framework Site. <https://site.mockito.org/> (последен достъп: 11.09.2026 г.)
- [23] Microsoft. Playwright Documentation. <https://playwright.dev/docs/intro> (последен достъп: 11.09.2026 г.)
- [24] Fielding, R. T. Architectural Styles and the Design of Network-based Software Architectures. Doctoral dissertation, University of California, Irvine, 2000.
<https://ics.uci.edu/~fielding/pubs/dissertation/top.htm> (последен достъп: 11.09.2026 г.)
- [25] Fowler, M. Patterns of Enterprise Application Architecture. Addison-Wesley, 2002.
- [26] OWASP Foundation. OWASP Top 10:2021. <https://owasp.org/Top10/> (последен достъп: 11.09.2026 г.)

- [27] Vocke, H. The Practical Test Pyramid. martinowler.com, 2018.
<https://martinowler.com/articles/practical-test-pyramid.html> (последен достъп:
11.09.2026 г.)

Приложение А. Изходен код на проекта

Пълният изходен код на проекта се предава като архив „12A_Илия_Влахов_Проект.zip“ заедно с файла README.md с инструкции за инсталиране и стартиране. В приложението са дадени структурата на проекта и пълният код на компонентите с алгоритмите и на конфигурацията за сигурност, описани в глава 4.

А.1. Структура на проекта

Ilko-Gramatikov/	
— pom.xml, mvnw, mvnw.cmd, .mvn/wrapper/	Maven проект и Maven Wrapper
— application-local.properties.example	примерни локални настройки
— dev.js, package.json	стартиране с една команда
— README.md	инструкции за стартиране
— src/main/java/com/ilko/tournament/	
— TournamentPlatformApplication.java	
— config/	AdminAccountInitializer, DemoDataInitializer, ParticipantSchemaGuard, TournamentFormatSchemaGuard, AuthenticationConfig
— controller/	AuthController, TournamentController, MatchController, NotificationController, AdminController
— dto/	записи за заявки и отговори
— entity/	AppUser, Tournament, Participant, TournamentGroup, TournamentMatch, Notification
— enums/	Role, TournamentFormat, TournamentStatus, MatchStatus, ParticipantStatus, NotificationType
— exception/	BusinessException, ConflictException, ResourceNotFoundException, UnauthorizedOperationException, GlobalExceptionHandler
— mapper/	TournamentMapper, UserMapper
— repository/	AppUserRepository, TournamentRepository, ParticipantRepository, TournamentGroupRepository, TournamentMatchRepository, NotificationRepository
— security/	SecurityConfig, AppUserDetailsService
— service/	BracketGenerator, DoubleEliminationGenerator, RoundRobinScheduler, RankingCalculator, TournamentStatisticsCalculator, интерфейси на услугите
— impl/	TournamentService, NotificationService, AuthService, UserAdminService
— src/main/resources/	application.properties, application-prod.properties
— src/test/java/com/ilko/tournament/	22 тестови класа
— frontend/	index.html, styles.css, app.js, server.js
— e2e/	api-smoke.js, verify.js, screenshots.js, server.js
— docs/	business-rules.md, testing-strategy.md, non-functional-review.md, migrations/, screenshots/

А.2. BracketGenerator.java

```
package com.ilko.tournament.service;
```

```

import com.ilko.tournament.entity.Participant;
import com.ilko.tournament.entity.Tournament;
import com.ilko.tournament.entity.TournamentMatch;
import com.ilko.tournament.enums.MatchStatus;
import org.springframework.stereotype.Component;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * Single-elimination bracket algorithm.
 *
 * <p>Participants are seeded in the order they are passed in (seed 1 first) and placed
with the standard
 * seeding pattern - for eight slots 1v8, 4v5, 2v7, 3v6 - so the two strongest seeds can
only meet in the
 * final. When the participant count is not a power of two, the unused seed positions
become BYEs. The
 * pattern pairs every BYE with one of the top seeds, so a BYE never meets another BYE
and every
 * participant plays a real opponent from round two onwards.</p>
 *
 * <p>Matches are addressed purely by position (round, match number): the feeders of
match k in round r
 * are matches 2k-1 and 2k of round r-1, and the winner of match k moves to match
ceil(k/2) of round r+1
 * (slot 1 for odd k, slot 2 for even k). The persisted {@code nextMatch} link mirrors
that structure, but
 * the algorithm never relies on it, so it behaves the same on new and on database-loaded
matches.</p>
 */
@Component
public class BracketGenerator {

    private static final Comparator<TournamentMatch> BRACKET_ORDER =
Comparator.comparingInt(TournamentMatch::getRoundNumber).thenComparingInt(TournamentMatch
::getMatchNumber);

    /** State of one side of a match while the bracket is being settled. */
    private enum Slot { FILLED, EMPTY, WAITING }

    private record Position(int round, int number) { }

    /** Builds every match of the bracket, links each match to its next match and settles
the BYEs. */
    public List<TournamentMatch> generate(Tournament tournament, List<Participant>
seeding) {
        if (seeding == null || seeding.size() < 2) {
            throw new IllegalArgumentException("Elimination bracket requires at least 2
participants.");
        }
        int bracketSize = nextPowerOfTwo(seeding.size());
        int totalRounds = Integer.numberOfTrailingZeros(bracketSize);
        int[] seeds = seedOrder(bracketSize);

        List<TournamentMatch> bracket = new ArrayList<>();
        int matchesInRound = bracketSize / 2;
        for (int round = 1; round <= totalRounds; round++, matchesInRound /= 2) {
            for (int number = 1; number <= matchesInRound; number++) {
                TournamentMatch match = new TournamentMatch();

```

```

        match.setTournament(tournament);
        match.setRoundNumber(round);
        match.setMatchNumber(number);
        match.setStatus(MatchStatus.PENDING);
        if (round == 1) {
            match.setParticipant1(seeded(seeding, seeds[2 * number - 2]));
            match.setParticipant2(seeded(seeding, seeds[2 * number - 1]));
        }
        bracket.add(match);
    }
}

Map<Position, TournamentMatch> index = index(bracket);
for (TournamentMatch match : bracket) {
    match.setNextMatch(index.get(nextPosition(match)));
}
propagate(bracket);
return bracket;
}

/**
 * Records that {@code completed} has been decided: moves its winner into the next
 * round and settles
 * whatever that unlocks. Returns the matches that became READY as a result.
 */
public List<TournamentMatch> advance(List<TournamentMatch> bracket, TournamentMatch
completed) {
    Position completedPosition = position(completed);
    List<TournamentMatch> current = new ArrayList<>(bracket.size());
    for (TournamentMatch match : bracket) {
        current.add(position(match).equals(completedPosition) ? completed : match);
    }
    placeWinner(index(current), completed);
    return propagate(current);
}

/**
 * Walks the bracket round by round and settles every unfinished match whose two
 * sides are known:
 * two participants make it READY, a participant facing an empty side wins by BYE,
 * and two empty sides
 * complete it without a winner. A side whose feeder match is still undecided is
 * WAITING, and the match
 * stays PENDING until that feeder is played. Returns the matches that became READY
 * during this call.
 */
public List<TournamentMatch> propagate(List<TournamentMatch> bracket) {
    Map<Position, TournamentMatch> index = index(bracket);
    List<TournamentMatch> ordered = new ArrayList<>(bracket);
    ordered.sort(BRACKET_ORDER);

    List<TournamentMatch> becameReady = new ArrayList<>();
    for (TournamentMatch match : ordered) {
        if (match.getStatus() == MatchStatus.COMPLETED) {
            continue;
        }
        Slot first = slot(match.getParticipant1(), feeder(index, match, 1));
        Slot second = slot(match.getParticipant2(), feeder(index, match, 2));
        if (first == Slot.WAITING || second == Slot.WAITING) {
            continue;
        }
        if (first == Slot.FILLED && second == Slot.FILLED) {
            if (match.getStatus() != MatchStatus.READY) {
                match.setStatus(MatchStatus.READY);
                becameReady.add(match);
            }
        }
    }
}

```

```

        }
        continue;
    }
    match.setWinner(first == Slot.FILLED ? match.getParticipant1() :
match.getParticipant2());
    match.setStatus(MatchStatus.COMPLETED);
    placeWinner(index, match);
}
return becameReady;
}

/**
 * Standard seeding order for a bracket of the given power-of-two size, listed by
bracket position.
 * Built by doubling: every seed s of the smaller bracket is followed by its opponent
size + 1 - s.
 */
static int[] seedOrder(int bracketSize) {
    int[] order = {1};
    while (order.length < bracketSize) {
        int size = order.length * 2;
        int[] expanded = new int[size];
        for (int i = 0; i < order.length; i++) {
            expanded[2 * i] = order[i];
            expanded[2 * i + 1] = size + 1 - order[i];
        }
        order = expanded;
    }
    return order;
}

private Slot slot(Participant participant, TournamentMatch feeder) {
    if (participant != null) {
        return Slot.FILLED;
    }
    // Round one has no feeder, so an empty side there is an unused seed position (a
BYE).
    if (feeder == null || feeder.getStatus() == MatchStatus.COMPLETED) {
        return Slot.EMPTY;
    }
    return Slot.WAITING;
}

private TournamentMatch feeder(Map<Position, TournamentMatch> index, TournamentMatch
match, int side) {
    if (match.getRoundNumber() == 1) {
        return null;
    }
    int feederNumber = side == 1 ? 2 * match.getMatchNumber() - 1 : 2 *
match.getMatchNumber();
    return index.get(new Position(match.getRoundNumber() - 1, feederNumber));
}

private void placeWinner(Map<Position, TournamentMatch> index, TournamentMatch match)
{
    TournamentMatch next = index.get(nextPosition(match));
    if (next == null) {
        return;
    }
    if (match.getMatchNumber() % 2 == 1) {
        next.setParticipant1(match.getWinner());
    } else {
        next.setParticipant2(match.getWinner());
    }
}
}

```

```

    private Position nextPosition(TournamentMatch match) {
        return new Position(match.getRoundNumber() + 1, (match.getMatchNumber() + 1) /
2);
    }

    private Position position(TournamentMatch match) {
        return new Position(match.getRoundNumber(), match.getMatchNumber());
    }

    private Map<Position, TournamentMatch> index(List<TournamentMatch> bracket) {
        Map<Position, TournamentMatch> index = new HashMap<>();
        for (TournamentMatch match : bracket) {
            index.put(position(match), match);
        }
        return index;
    }

    private Participant seeded(List<Participant> seeding, int seed) {
        return seed <= seeding.size() ? seeding.get(seed - 1) : null;
    }

    private int nextPowerOfTwo(int n) {
        int highest = Integer.highestOneBit(n);
        return highest == n ? n : highest << 1;
    }
}

```

A.3. DoubleEliminationGenerator.java

```

package com.ilko.tournament.service;

import com.ilko.tournament.entity.Participant;
import com.ilko.tournament.entity.Tournament;
import com.ilko.tournament.entity.TournamentMatch;
import com.ilko.tournament.enums.BracketSide;
import com.ilko.tournament.enums.MatchStatus;
import org.springframework.stereotype.Component;

import java.util.ArrayList;
import java.util.Comparator;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * Double-elimination bracket algorithm: a participant is out only after losing twice.
 *
 * <p>The winners bracket is seeded exactly like the single-elimination one (see
 * {@link BracketGenerator#seedOrder(int)}), but its losers are not eliminated - they
 * drop into a
 * losers bracket and can fight their way back to the grand final. For a bracket of size
 * 2^n the
 * structure is: n winners rounds, 2n-2 losers rounds and the grand final, which is 2 *
 * 2^n - 2
 * matches in total.</p>
 *
 * <p>The losers bracket alternates between two kinds of round. Odd rounds pair survivors
 * of the
 * losers bracket against each other; even rounds put those survivors against the players
 * just

```

```

    * knocked out of the winners bracket, so nobody meets a fresh winners-bracket loser
    twice in a
    * row. Losers of winners round 1 start in losers round 1, and losers of winners round r
    > 1
    * enter at losers round 2(r-1).</p>
    *
    * <p>Round numbers are allocated so that they stay unique per tournament and every link
    points
    * forward: winners rounds are 1..n, losers rounds n+1..3n-2, the grand final is 3n-1 and
    the
    * optional deciding rematch 3n. Because of that, settling the bracket is a single pass
    in
    * ascending round order, and matches can be persisted in descending round order while
    the
    * {@code next_match_id} / {@code next_loser_match_id} foreign keys still resolve.</p>
    *
    * <p>Unlike the single-elimination generator, which derives everything from positions,
    this one
    * follows the stored {@code nextMatch}/{@code nextLoserMatch} links: the losers bracket
    has two
    * different feeding patterns, so an explicit link is clearer than positional arithmetic.
    Links are
    * resolved by id against the passed-in list, so a lazily-loaded proxy is never
    initialised and the
    * algorithm behaves the same on freshly generated and on database-loaded matches.</p>
    */
@Component
public class DoubleEliminationGenerator {

    static final Comparator<TournamentMatch> BRACKET_ORDER =
Comparator.comparingInt(TournamentMatch::getRoundNumber).thenComparingInt(TournamentMatch
::getMatchNumber);

    /** State of one side of a match while the bracket is being settled. */
    private enum Slot { FILLED, EMPTY, WAITING }

    private record Position(int round, int number) { }

    /** One side (1 or 2) of one match. */
    private record SideKey(int round, int number, int slot) { }

    /** The match a side is waiting for, and whether it is that match's winner or its
    loser. */
    private record Source(TournamentMatch match, boolean loser) { }

    /** Builds every match, links winners and losers to where they go next and settles
    the BYEs. */
    public List<TournamentMatch> generate(Tournament tournament, List<Participant>
seeding) {
        if (seeding == null || seeding.size() < 2) {
            throw new IllegalArgumentException("Double-elimination bracket requires at
least 2 participants.");
        }
        int bracketSize = nextPowerOfTwo(seeding.size());
        int winnersRounds = Integer.numberOfTrailingZeros(bracketSize);
        int losersRounds = losersRounds(winnersRounds);
        int[] seeds = BracketGenerator.seedOrder(bracketSize);

        Map<Position, TournamentMatch> index = new HashMap<>();
        List<TournamentMatch> bracket = new ArrayList<>();

        for (int round = 1; round <= winnersRounds; round++) {

```

```

        for (int number = 1, count = 1 << (winnersRounds - round); number <= count;
number++) {
            TournamentMatch match = add(bracket, index, tournament,
BracketSide.WINNERS, round, number);
            if (round == 1) {
                match.setParticipant1(seeded(seeding, seeds[2 * number - 2]));
                match.setParticipant2(seeded(seeding, seeds[2 * number - 1]));
            }
        }
        for (int round = 1; round <= losersRounds; round++) {
            for (int number = 1, count = losersMatches(winnersRounds, round); number <=
count; number++) {
                add(bracket, index, tournament, BracketSide.LOSERS, winnersRounds +
round, number);
            }
        }
        int grandFinalRound = winnersRounds + losersRounds + 1;
        add(bracket, index, tournament, BracketSide.GRAND_FINAL, grandFinalRound, 1);
        if (tournament.isGrandFinalReset()) {
            add(bracket, index, tournament, BracketSide.GRAND_FINAL, grandFinalRound + 1,
1);
        }

        link(index, winnersRounds, losersRounds, grandFinalRound);
        propagate(bracket);
        return bracket;
    }

    /**
     * Records that {@code completed} has been decided: moves its winner and its loser to
wherever
     * they go next and settles whatever that unlocks. Returns the matches that became
READY.
     */
    public List<TournamentMatch> advance(List<TournamentMatch> bracket, TournamentMatch
completed) {
        Position completedPosition = position(completed);
        List<TournamentMatch> current = new ArrayList<>(bracket.size());
        for (TournamentMatch match : bracket) {
            current.add(position(match).equals(completedPosition) ? completed : match);
        }
        placeOutcome(byId(current), completed);
        return propagate(current);
    }

    /**
     * Walks the bracket in ascending round order and settles every unfinished match
whose two sides
     * are known: two participants make it READY, a participant facing an empty side wins
by BYE, and
     * two empty sides complete it without a winner. A side still waiting for a feeder
keeps the match
     * PENDING. Returns the matches that became READY during this call.
     */
    public List<TournamentMatch> propagate(List<TournamentMatch> bracket) {
        List<TournamentMatch> ordered = new ArrayList<>(bracket);
        ordered.sort(BRACKET_ORDER);
        Map<SideKey, Source> sources = sources(bracket);
        Map<Long, TournamentMatch> byId = byId(bracket);
        TournamentMatch decider = decider(ordered);

        List<TournamentMatch> becameReady = new ArrayList<>();
        for (TournamentMatch match : ordered) {
            if (match.getStatus() == MatchStatus.COMPLETED) {

```

```

        continue;
    }
    if (match == decider) {
        settleDecider(decider, grandFinal(ordered), becameReady);
        continue;
    }
    Slot first = slot(match.getParticipant1(), sources.get(sideKey(match, 1)));
    Slot second = slot(match.getParticipant2(), sources.get(sideKey(match, 2)));
    if (first == Slot.WAITING || second == Slot.WAITING) {
        continue;
    }
    if (first == Slot.FILLED && second == Slot.FILLED) {
        if (match.getStatus() != MatchStatus.READY) {
            match.setStatus(MatchStatus.READY);
            becameReady.add(match);
        }
        continue;
    }
    match.setWinner(first == Slot.FILLED ? match.getParticipant1() :
match.getParticipant2());
    match.setStatus(MatchStatus.COMPLETED);
    placeOutcome(byId, match);
}
return becameReady;
}

/** Number of losers rounds: none for a two-player bracket, otherwise 2n-2. */
static int losersRounds(int winnersRounds) {
    return winnersRounds < 2 ? 0 : 2 * winnersRounds - 2;
}

/** Losers round r holds 2^(n-1-ceil(r/2)) matches, halving after every pairing
round. */
static int losersMatches(int winnersRounds, int losersRound) {
    return 1 << (winnersRounds - 1 - (losersRound + 1) / 2);
}

// --- structure ---

private void link(Map<Position, TournamentMatch> index, int winnersRounds, int
losersRounds, int grandFinalRound) {
    TournamentMatch grandFinal = index.get(new Position(grandFinalRound, 1));

    for (int round = 1; round <= winnersRounds; round++) {
        for (int number = 1, count = 1 << (winnersRounds - round); number <= count;
number++) {
            TournamentMatch match = index.get(new Position(round, number));
            if (round < winnersRounds) {
                setNext(match, index.get(new Position(round + 1, (number + 1) / 2)),
side(number));
            } else {
                setNext(match, grandFinal, 1);
            }
            if (losersRounds == 0) {
                // Two players: the loser of the only match goes straight to the
grand final.
                setLoserNext(match, grandFinal, 2);
            } else if (round == 1) {
                setLoserNext(match, index.get(new Position(winnersRounds + 1, (number
+ 1) / 2)), side(number));
            } else {
                setLoserNext(match, index.get(new Position(winnersRounds + 2 * (round
- 1), number)), 2);
            }
        }
    }
}

```

```

    }
    }
    for (int round = 1; round <= losersRounds; round++) {
        for (int number = 1, count = losersMatches(winnersRounds, round); number <=
count; number++) {
            TournamentMatch match = index.get(new Position(winnersRounds + round,
number));
            if (round == losersRounds) {
                setNext(match, grandFinal, 2);
            } else if ((round + 1) % 2 == 0) {
                // The next round also takes a winners-bracket loser, on side 2.
                setNext(match, index.get(new Position(winnersRounds + round + 1,
number)), 1);
            } else {
                setNext(match, index.get(new Position(winnersRounds + round + 1,
(number + 1) / 2)), side(number));
            }
        }
    }
}

private TournamentMatch add(List<TournamentMatch> bracket, Map<Position,
TournamentMatch> index,
                        Tournament tournament, BracketSide side, int round, int
number) {
    TournamentMatch match = new TournamentMatch();
    match.setTournament(tournament);
    match.setBracket(side);
    match.setRoundNumber(round);
    match.setMatchNumber(number);
    match.setStatus(MatchStatus.PENDING);
    bracket.add(match);
    index.put(new Position(round, number), match);
    return match;
}

private void setNext(TournamentMatch match, TournamentMatch next, int slot) {
    match.setNextMatch(next);
    match.setNextMatchSlot(slot);
}

private void setLoserNext(TournamentMatch match, TournamentMatch next, int slot) {
    match.setNextLoserMatch(next);
    match.setNextLoserSlot(slot);
}

private int side(int number) {
    return number % 2 == 1 ? 1 : 2;
}

// --- settling ---

/**
 * The deciding rematch is not fed by links: it is played only when the losers-
bracket finalist
 * wins the grand final, and is otherwise marked complete without participants, so
the tournament
 * can finish while the bracket still shows that the match existed.
 */
private void settleDecider(TournamentMatch decider, TournamentMatch grandFinal,
List<TournamentMatch> becameReady) {
    if (grandFinal == null || grandFinal.getStatus() != MatchStatus.COMPLETED) {
        return;
    }
}

```

```

        if (wonBy(grandFinal, grandFinal.getParticipant2())) {
            decider.setParticipant1(grandFinal.getParticipant1());
            decider.setParticipant2(grandFinal.getParticipant2());
            if (decider.getStatus() != MatchStatus.READY) {
                decider.setStatus(MatchStatus.READY);
                becameReady.add(decider);
            }
            return;
        }
        decider.setStatus(MatchStatus.COMPLETED); // not needed: the unbeaten finalist
won the grand final
    }

    private Slot slot(Participant participant, Source source) {
        if (participant != null) {
            return Slot.FILLED;
        }
        // No feeder at all is an unused seed position in winners round 1, i.e. a BYE.
        if (source == null || source.match().getStatus() == MatchStatus.COMPLETED) {
            return Slot.EMPTY;
        }
        return Slot.WAITING;
    }

    private void placeOutcome(Map<Long, TournamentMatch> byId, TournamentMatch match) {
        TournamentMatch next = resolve(byId, match.getNextMatch());
        if (next != null && match.getWinner() != null) {
            setSide(next, match.getNextMatchSlot(), match.getWinner());
        }
        TournamentMatch loserNext = resolve(byId, match.getNextLoserMatch());
        Participant loser = loserOf(match);
        if (loserNext != null && loser != null) {
            setSide(loserNext, match.getNextLoserSlot(), loser);
        }
    }

    private void setSide(TournamentMatch match, Integer slot, Participant participant) {
        if (slot != null && slot == 2) {
            match.setParticipant2(participant);
        } else {
            match.setParticipant1(participant);
        }
    }

    /** The beaten participant, or null when the match was a BYE, a draw or is still
open. */
    private Participant loserOf(TournamentMatch match) {
        if (match.getWinner() == null || match.getParticipant1() == null ||
match.getParticipant2() == null) {
            return null;
        }
        return wonBy(match, match.getParticipant1()) ? match.getParticipant2() :
match.getParticipant1();
    }

    private boolean wonBy(TournamentMatch match, Participant participant) {
        return same(match.getWinner(), participant);
    }

    /** Identity first, id second: works for an unsaved bracket and for one loaded as
proxies. */
    private boolean same(Participant a, Participant b) {
        if (a == null || b == null) {
            return false;
        }
    }

```

```

    }
    return a == b || (a.getId() != null && a.getId().equals(b.getId()));
}

/** Which match (and which of its outcomes) every side of every match is waiting for.
 */
private Map<SideKey, Source> sources(List<TournamentMatch> bracket) {
    Map<Long, TournamentMatch> byId = byId(bracket);
    Map<SideKey, Source> sources = new HashMap<>();
    for (TournamentMatch match : bracket) {
        TournamentMatch next = resolve(byId, match.getNextMatch());
        if (next != null) {
            sources.put(sideKey(next, slotOf(match.getNextMatchSlot())), new
Source(match, false));
        }
        TournamentMatch loserNext = resolve(byId, match.getNextLoserMatch());
        if (loserNext != null) {
            sources.put(sideKey(loserNext, slotOf(match.getNextLoserSlot())), new
Source(match, true));
        }
    }
    return sources;
}

/**
 * The instance of {@code link} that belongs to this list. Reading only the id never
initialises a
 * lazy proxy, so a database-loaded bracket is walked without extra queries.
 */
private TournamentMatch resolve(Map<Long, TournamentMatch> byId, TournamentMatch
link) {
    if (link == null) {
        return null;
    }
    if (link.getId() == null) {
        return link; // freshly generated bracket: the link already points at the
right object
    }
    return byId.get(link.getId());
}

private Map<Long, TournamentMatch> byId(List<TournamentMatch> bracket) {
    Map<Long, TournamentMatch> byId = new HashMap<>();
    for (TournamentMatch match : bracket) {
        if (match.getId() != null) {
            byId.put(match.getId(), match);
        }
    }
    return byId;
}

/** Sides are addressed by position so that new and database-loaded matches key the
same way. */
private SideKey sideKey(TournamentMatch match, int slot) {
    return new SideKey(match.getRoundNumber(), match.getMatchNumber(), slot);
}

private int slotOf(Integer slot) {
    return slot != null && slot == 2 ? 2 : 1;
}

private TournamentMatch grandFinal(List<TournamentMatch> ordered) {
    return ordered.stream().filter(m -> m.getBracket() ==
BracketSide.GRAND_FINAL).findFirst().orElse(null);
}

```

```

    }

    /** The deciding rematch, i.e. the second grand-final match, when the tournament has
    one. */
    private TournamentMatch decider(List<TournamentMatch> ordered) {
        List<TournamentMatch> finals = ordered.stream().filter(m -> m.getBracket() ==
BracketSide.GRAND_FINAL).toList();
        return finals.size() > 1 ? finals.get(1) : null;
    }

    private Position position(TournamentMatch match) {
        return new Position(match.getRoundNumber(), match.getMatchNumber());
    }

    private Participant seeded(List<Participant> seeding, int seed) {
        return seed <= seeding.size() ? seeding.get(seed - 1) : null;
    }

    private int nextPowerOfTwo(int n) {
        int highest = Integer.highestOneBit(n);
        return highest == n ? n : highest << 1;
    }
}

```

A.4. RoundRobinScheduler.java

```

package com.ilko.tournament.service;

import org.springframework.stereotype.Component;

import java.util.ArrayList;
import java.util.List;

/**
 * Round-robin schedule for one group, built with the circle method (the basis of Berger
 tables).
 *
 * <p>Positions 0..n-1 are placed on a ring; in every round position i plays position n-
 1-i, then every
 * position except the first rotates one step. After n-1 rounds each pair has met exactly
 once and nobody
 * plays twice in the same round. With an odd participant count an extra "rest" position
 is added, and
 * whoever is paired with it sits that round out.</p>
 */
@Component
public class RoundRobinScheduler {

    /**
     * Returns the rounds of the schedule; each round lists pairs of indexes into the
 participant list,
     * the lower index first.
     */
    public List<List<int[]>> schedule(int participantCount) {
        if (participantCount < 2) {
            return List.of();
        }
        int slots = participantCount % 2 == 0 ? participantCount : participantCount + 1;
        int[] ring = new int[slots];
        for (int i = 0; i < slots; i++) {
            ring[i] = i;

```

```

    }

    List<List<int[]>> rounds = new ArrayList<>();
    for (int round = 0; round < slots - 1; round++) {
        List<int[]> pairs = new ArrayList<>();
        for (int i = 0; i < slots / 2; i++) {
            int a = ring[i];
            int b = ring[slots - 1 - i];
            if (a < participantCount && b < participantCount) {
                pairs.add(new int[]{Math.min(a, b), Math.max(a, b)});
            }
        }
        rounds.add(pairs);

        int last = ring[slots - 1];
        System.arraycopy(ring, 1, ring, 2, slots - 2);
        ring[1] = last;
    }
    return rounds;
}
}

```

A.5. RankingCalculator.java

```

package com.ilko.tournament.service;

import com.ilko.tournament.dto.RankingResponse;
import com.ilko.tournament.entity.Participant;
import com.ilko.tournament.entity.Tournament;
import com.ilko.tournament.entity.TournamentMatch;
import com.ilko.tournament.enums.MatchStatus;
import com.ilko.tournament.enums.TournamentFormat;
import org.springframework.stereotype.Component;

import java.util.Comparator;
import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Set;

/**
 * Computes participant rankings/statistics purely from real, persisted match results.
 * This is the single source of ranking logic for both ELIMINATION and GROUPS tournaments
 * -
 * it must never be duplicated or re-implemented elsewhere.
 *
 * GROUPS ranking order: most wins first, then best score difference, then participant
 * id.
 * Points (3 per win, 1 per draw, 0 per loss) are reported alongside the standings but
 * are not
 * themselves the sort key - wins remain the primary ranking criterion, unchanged by
 * this.
 *
 * ELIMINATION and DOUBLE_ELIMINATION ranking order additionally sorts first by how far
 * each participant progressed
 * in the bracket (round reached before elimination, or "champion" for the winner of the
 * final),
 * before falling back to the same wins / score-difference / id tie-break used for
 * GROUPS. This
 * progression tier is derived entirely from the round number and winner already stored
 * on each
 */

```

```

    * match - it is not a second, separately-tracked ranking value.
    */
@Component
public class RankingCalculator {

    private static final class Stats {
        String name;
        int played;
        int wins;
        int draws;
        int losses;
        int scoreFor;
        int scoreAgainst;
    }

    public List<RankingResponse> calculate(Tournament tournament, List<TournamentMatch>
matches) {
        Map<Long, Stats> stats = new LinkedHashMap<>();
        for (Participant participant : tournament.getParticipants()) {
            Stats s = new Stats();
            s.name = participant.getName();
            stats.put(participant.getId(), s);
        }

        for (TournamentMatch match : matches) {
            // Only fully-recorded, real results count. Byes (no scores) and unfinished
matches are correctly excluded.
            // A draw (real scores, no winner) DOES count - see the win/draw/loss tally
below.
            if (match.getStatus() != MatchStatus.COMPLETED || match.getScore1() == null
|| match.getScore2() == null) {
                continue;
            }
            Stats first = stats.get(match.getParticipant1().getId());
            Stats second = stats.get(match.getParticipant2().getId());
            if (first == null || second == null) continue; // defensive: ignore results
for participants no longer in the tournament

            first.played++;
            second.played++;
            first.scoreFor += match.getScore1();
            first.scoreAgainst += match.getScore2();
            second.scoreFor += match.getScore2();
            second.scoreAgainst += match.getScore1();

            if (match.getWinner() == null) {
                first.draws++;
                second.draws++;
            } else if (match.getWinner().getId().equals(match.getParticipant1().getId()))
{
                first.wins++;
                second.losses++;
            } else {
                second.wins++;
                first.losses++;
            }
        }

        // Both knockout formats rank by how far a participant got. Double-elimination
round numbers are
        // allocated so that losers rounds follow the winners rounds, so "the later you
lost, the better you
        // placed" holds there too - a participant is only out after their second loss,
in the losers bracket.

```

```

        boolean isElimination = tournament.getFormat() != TournamentFormat.GROUPS;
        Map<Long, Integer> progression = isElimination ?
eliminationProgression(stats.keySet(), matches) : Map.of();

        List<Long> ordered = stats.keySet().stream()
            .sorted(Comparator
                .comparingInt((Long id) -> isElimination ?
progression.getDefault(id, 1) : 0).reversed()
                .thenComparing(Comparator.comparingInt((Long id) ->
stats.get(id).wins).reversed())
                .thenComparing(Comparator.comparingInt((Long id) ->
scoreDifference(stats.get(id))).reversed())
                .thenComparingLong(id -> id))
            .toList();

        return java.util.stream.IntStream.range(0, ordered.size()).mapToObj(index -> {
            Long participantId = ordered.get(index);
            Stats s = stats.get(participantId);
            int scoreDiff = scoreDifference(s);
            return new RankingResponse(participantId, s.name, s.played, s.wins, s.draws,
s.losses, scoreDiff, s.wins * 3 + s.draws, index + 1);
        }).toList();
    }

    /**
     * How far each participant progressed in a knockout bracket, expressed as a single
     * comparable
     * tier: 1 = hasn't played/been eliminated yet, N = reached round N but lost there,
     * (highest round + 1) = won the final (tournament champion). Derived only from each
     * match's
     * existing roundNumber/winner - byes count as real progression (the bye winner
     * genuinely
     * advances), matching how BracketGenerator already treats them.
     */
    private Map<Long, Integer> eliminationProgression(Set<Long> participantIds,
List<TournamentMatch> matches) {
        int totalRounds =
matches.stream().mapToInt(TournamentMatch::getRoundNumber).max().orElse(0);
        Map<Long, Integer> progression = new HashMap<>();
        for (Long id : participantIds) progression.put(id, 1);

        for (TournamentMatch match : matches) {
            if (match.getStatus() != MatchStatus.COMPLETED || match.getWinner() == null)
continue;
            int round = match.getRoundNumber();
            markProgression(progression, participantIds, match.getParticipant1(),
match.getWinner(), round, totalRounds);
            markProgression(progression, participantIds, match.getParticipant2(),
match.getWinner(), round, totalRounds);
        }
        return progression;
    }

    private void markProgression(Map<Long, Integer> progression, Set<Long>
participantIds, Participant participant,
                                Participant winner, int round, int totalRounds) {
        if (participant == null || !participantIds.contains(participant.getId())) return;
        boolean won = winner.getId().equals(participant.getId());
        int tier = won ? round + 1 : round; // survived into round+1, or eliminated at
        exactly this round
        progression.merge(participant.getId(), tier, Math::max);
    }

    private int scoreDifference(Stats s) { return s.scoreFor - s.scoreAgainst; }

```

```
}
```

A.6. SecurityConfig.java

```
package com.ilko.tournament.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.web.cors.CorsConfiguration;
import org.springframework.web.cors.CorsConfigurationSource;
import org.springframework.web.cors.UrlBasedCorsConfigurationSource;
import java.util.List;
import org.springframework.security.config.annotation.method.configuration.EnableMethodSecurity;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.config.http.SessionCreationPolicy;
import jakarta.servlet.http.HttpServletRequest;
import java.nio.charset.StandardCharsets;
import java.time.LocalDateTime;

@Configuration
@EnableMethodSecurity
public class SecurityConfig {

    @Bean
    SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
        return http
            .csrf(csrf -> csrf.disable())
            .cors(cors -> {})
            .sessionManagement(session ->
                session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .exceptionHandling(exceptions -> exceptions
                .authenticationEntryPoint((request, response, exception) ->
                    writeError(response, request.getRequestURI(), HttpServletResponse.SC_UNAUTHORIZED,
                        "Authentication is required"))
                .accessDeniedHandler((request, response, exception) ->
                    writeError(response, request.getRequestURI(), HttpServletResponse.SC_FORBIDDEN, "Access
                        is denied")))
            .authorizeHttpRequests(authorize -> authorize
                .requestMatchers("/api/auth/**", "/error").permitAll()
                .requestMatchers("/api/admin/**").hasRole("ADMINISTRATOR")
                .anyRequest().authenticated())
            .httpBasic(basic -> {})
            .build();
    }

    private void writeError(HttpServletRequest response, String path, int status, String
        message) throws java.io.IOException {
        response.setStatus(status);
        response.setContentType("application/json");
        response.setCharacterEncoding(StandardCharsets.UTF_8.name());
        String error = status == HttpServletResponse.SC_UNAUTHORIZED ? "Unauthorized" :
            "Forbidden";
        response.getWriter().write("{\"timestamp\":\"" + LocalDateTime.now() +
            "\",\"status\":\"" + status
                + "\",\"error\":\"" + error + "\",\"message\":\"" + message
                + "\",\"path\":\"" + path.replace("\\", "\\\\").replace("\"", "\\\"") +
            "\"}");
    }
}
```

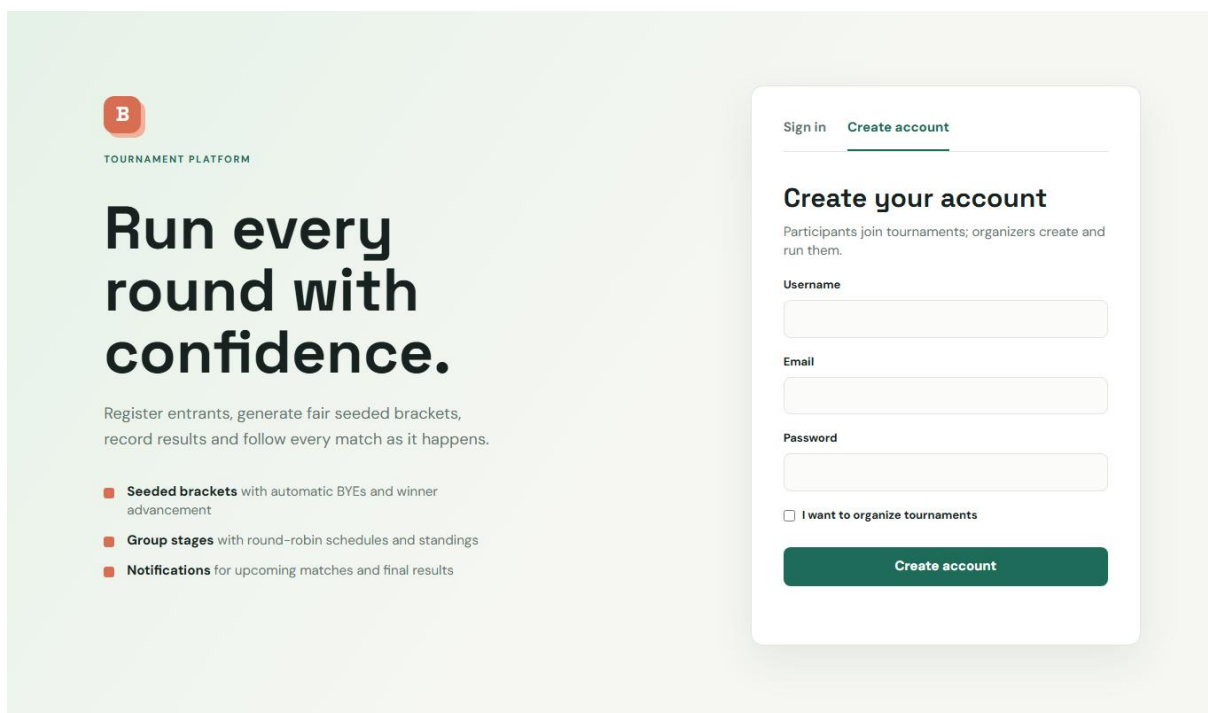
```

@Bean
CorsConfigurationSource corsConfigurationSource() {
    CorsConfiguration configuration = new CorsConfiguration();
    configuration.setAllowedOriginPatterns(List.of("http://localhost:*]",
"http://127.0.0.1:*]"));
    configuration.setAllowedMethods(List.of("GET", "POST", "PUT", "DELETE",
"OPTIONS"));
    configuration.setAllowedHeaders(List.of("Authorization", "Content-Type"));
    configuration.setAllowCredentials(true);
    UrlBasedCorsConfigurationSource source = new UrlBasedCorsConfigurationSource();
    source.registerCorsConfiguration("/**", configuration);
    return source;
}

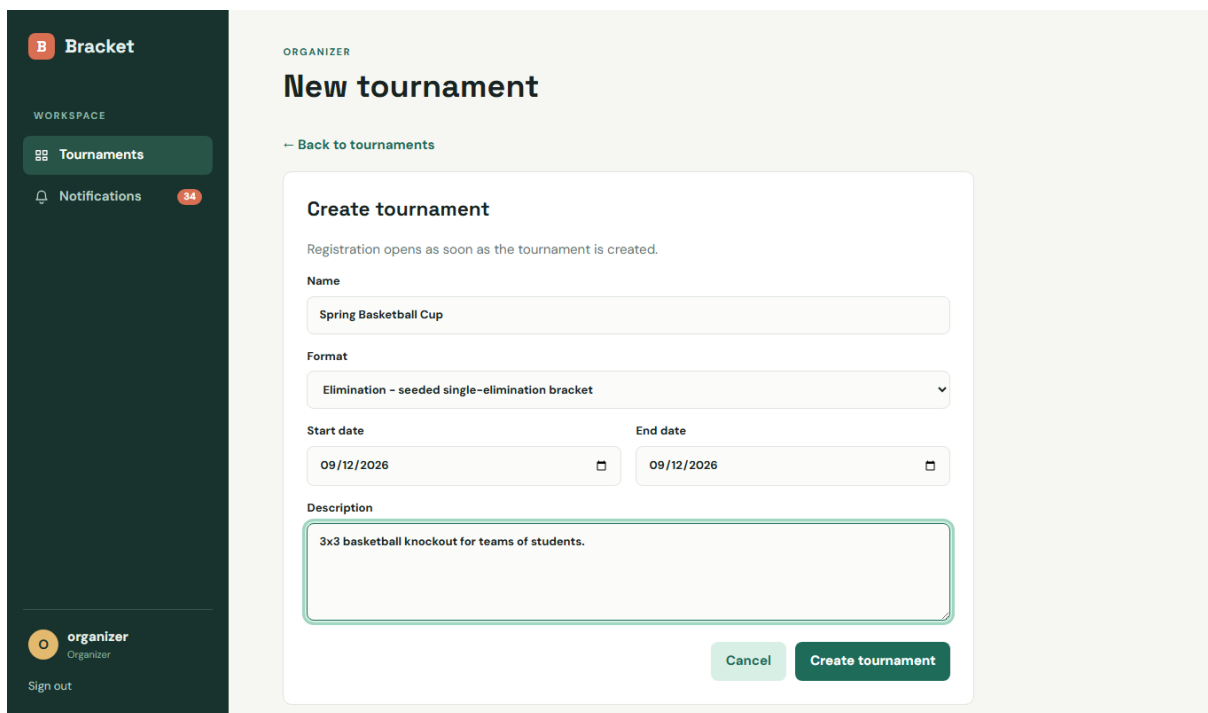
@Bean
PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
}

```

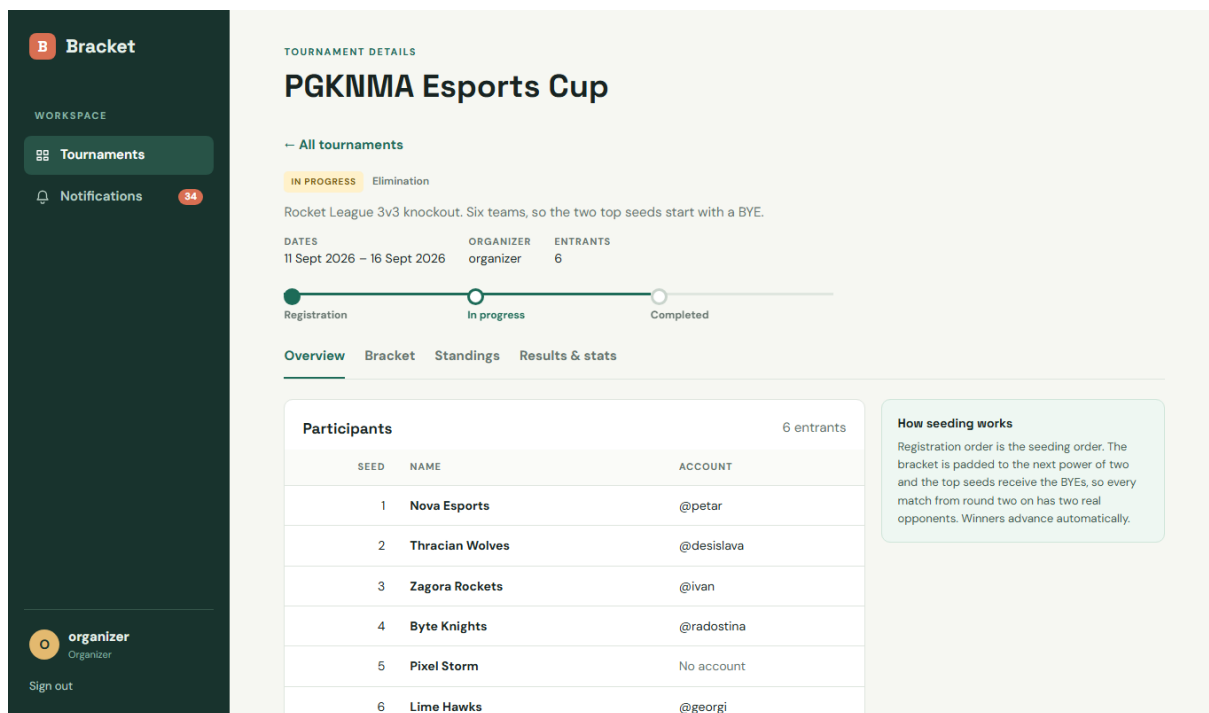
Приложение Б. Допълнителни снимки на екрана



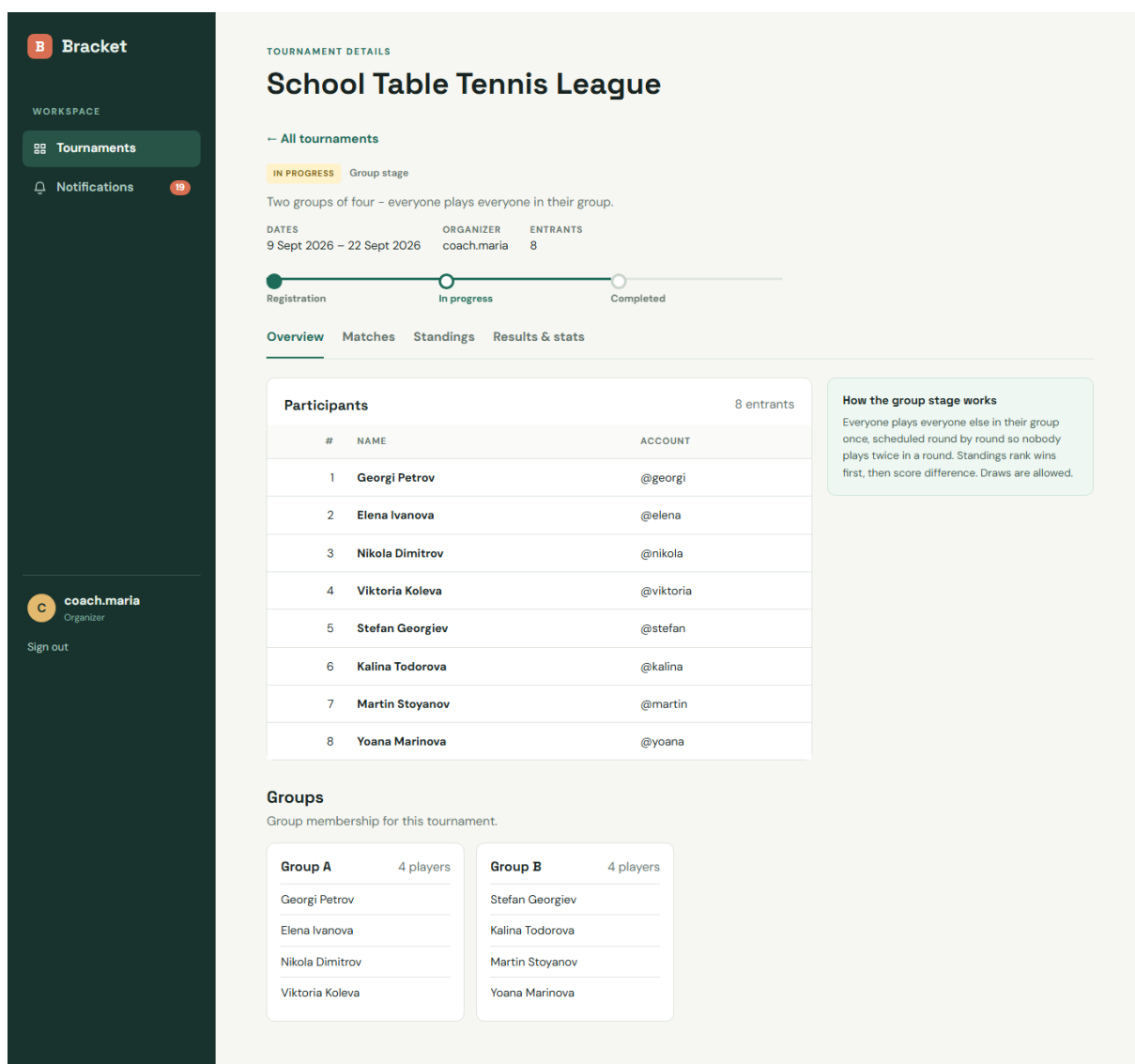
Фигура 19. Регистрация на нов потребител



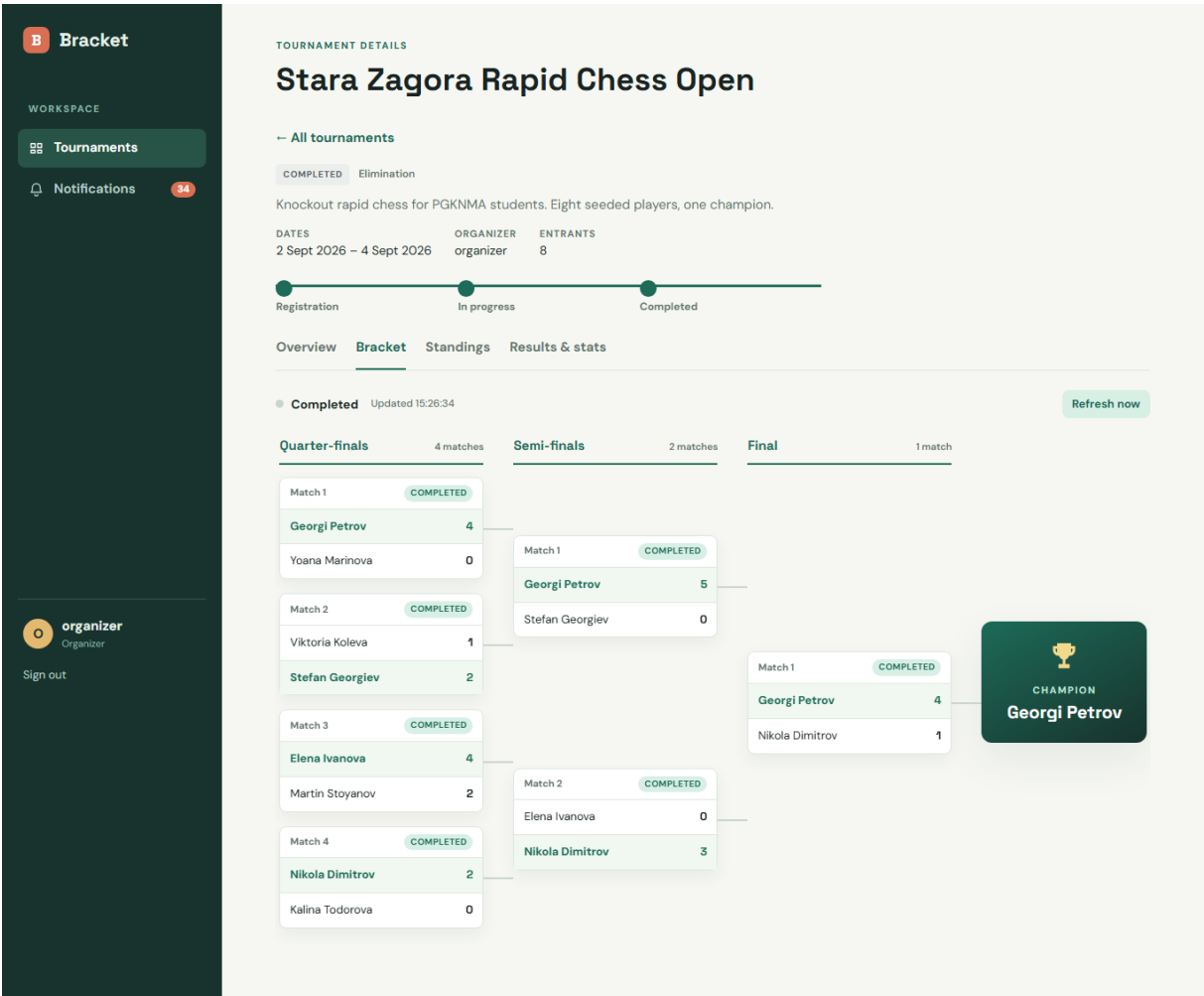
Фигура 20. Създаване на нов турнир



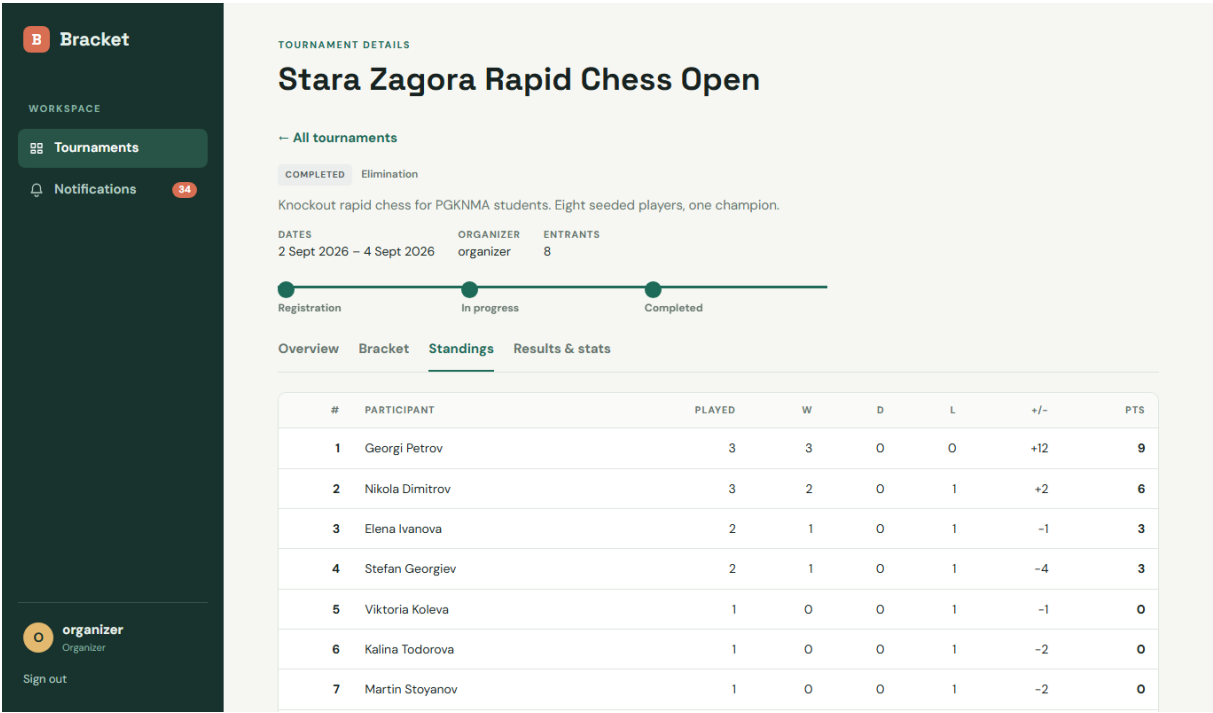
Фигура 21. Преглед на турнир в ход и неговите участници



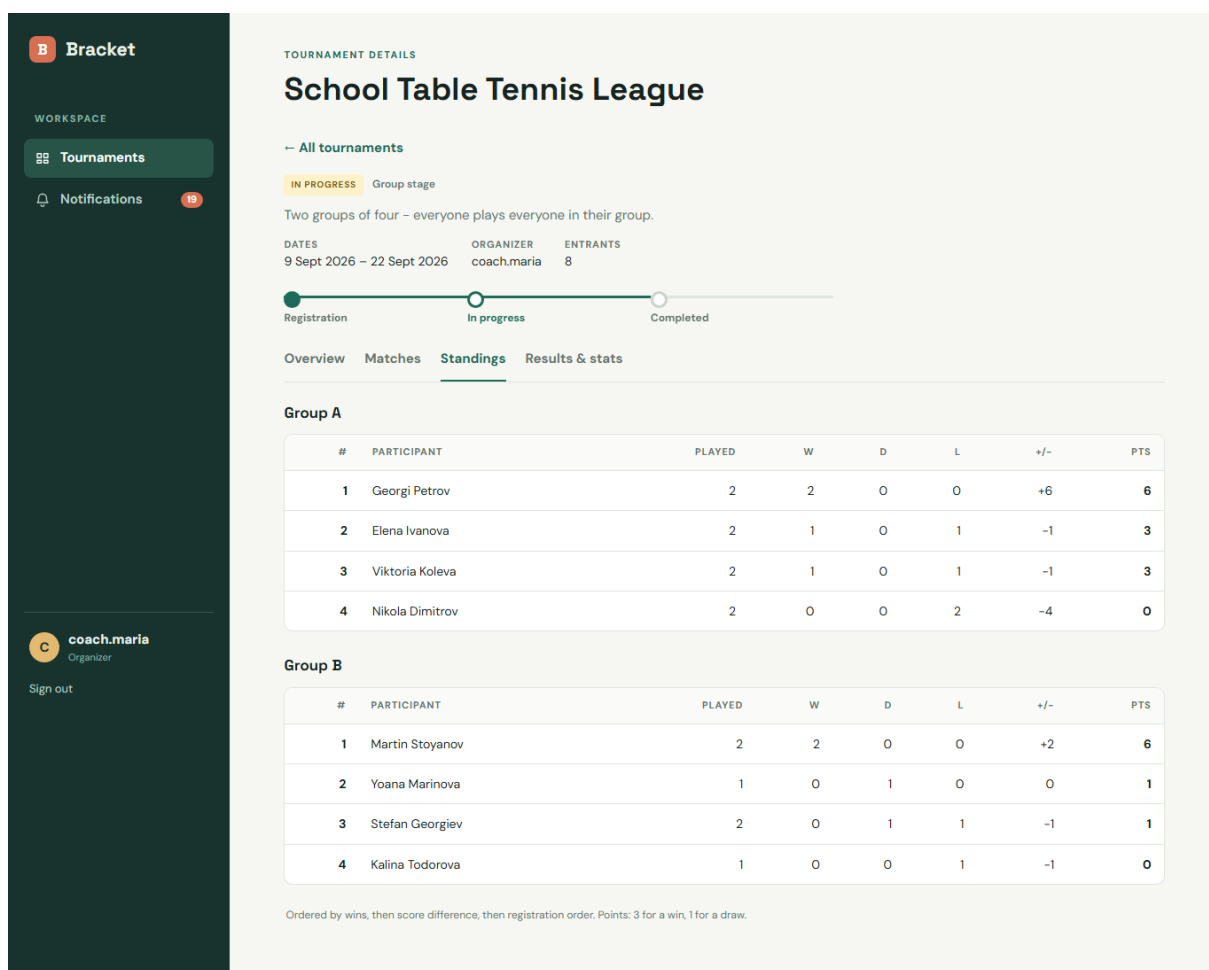
Фигура 22. Участници и разпределение по групи

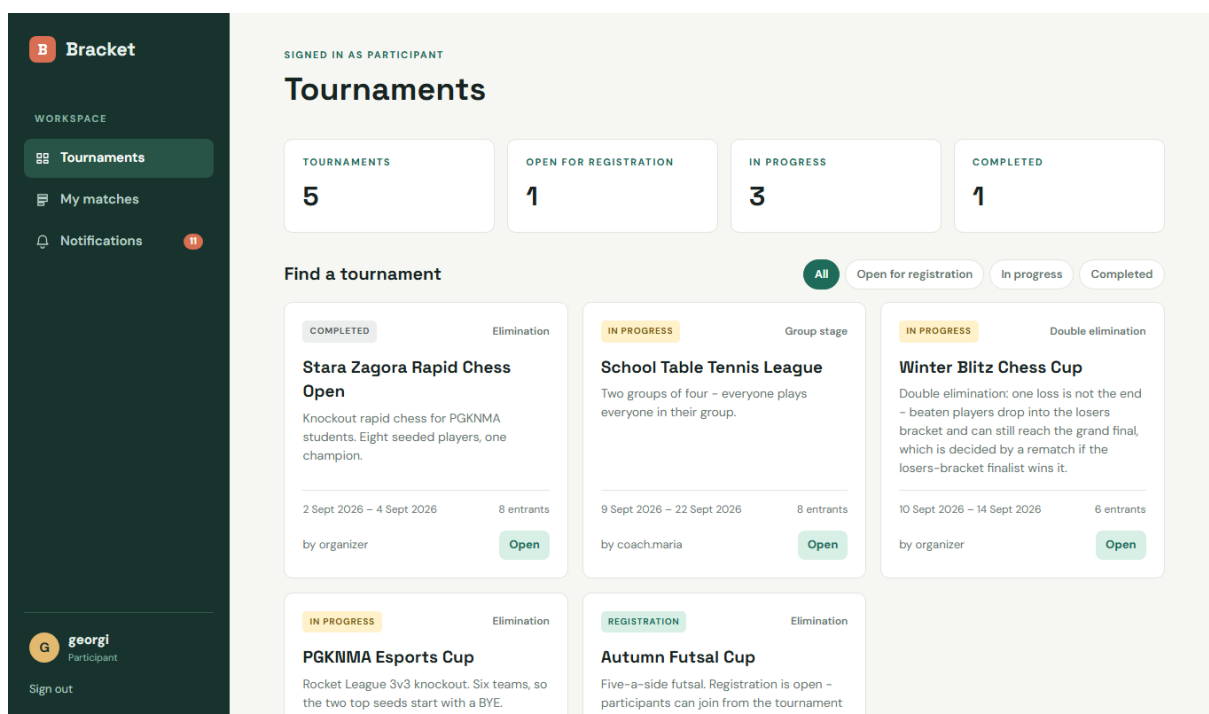


Фигура 23. Приключила схема с изведен шампион

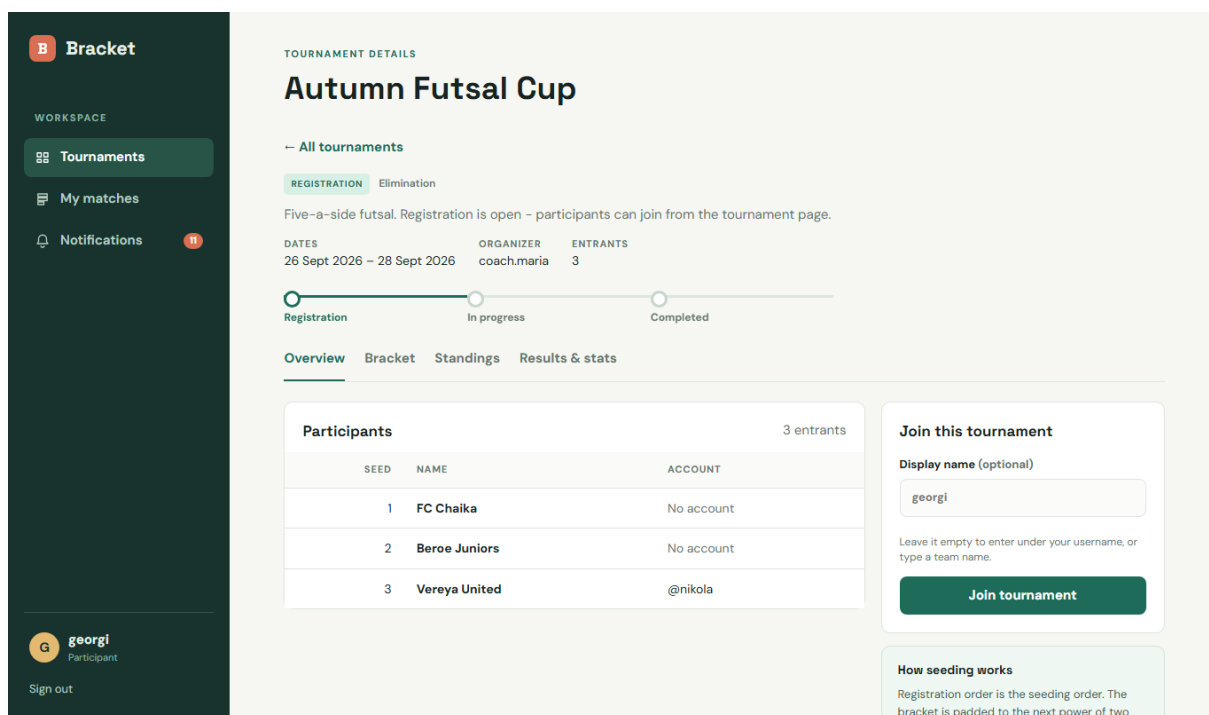


Фигура 24. Класиране на турнир с директни елиминации

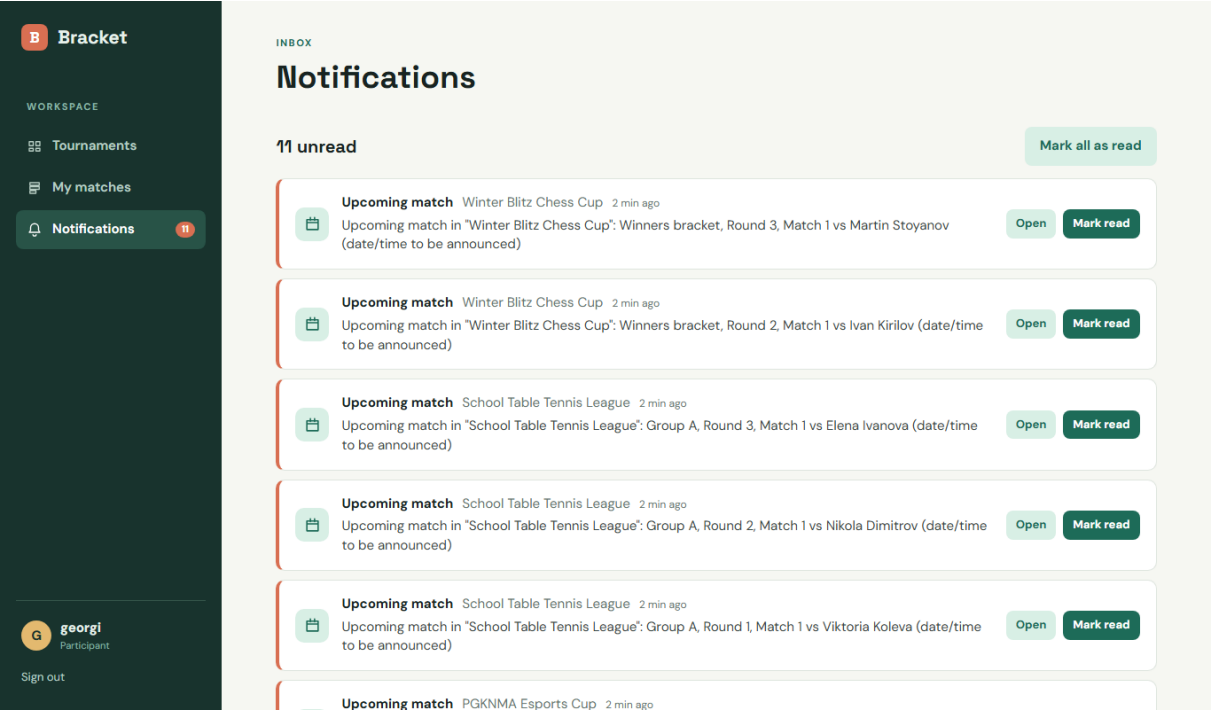




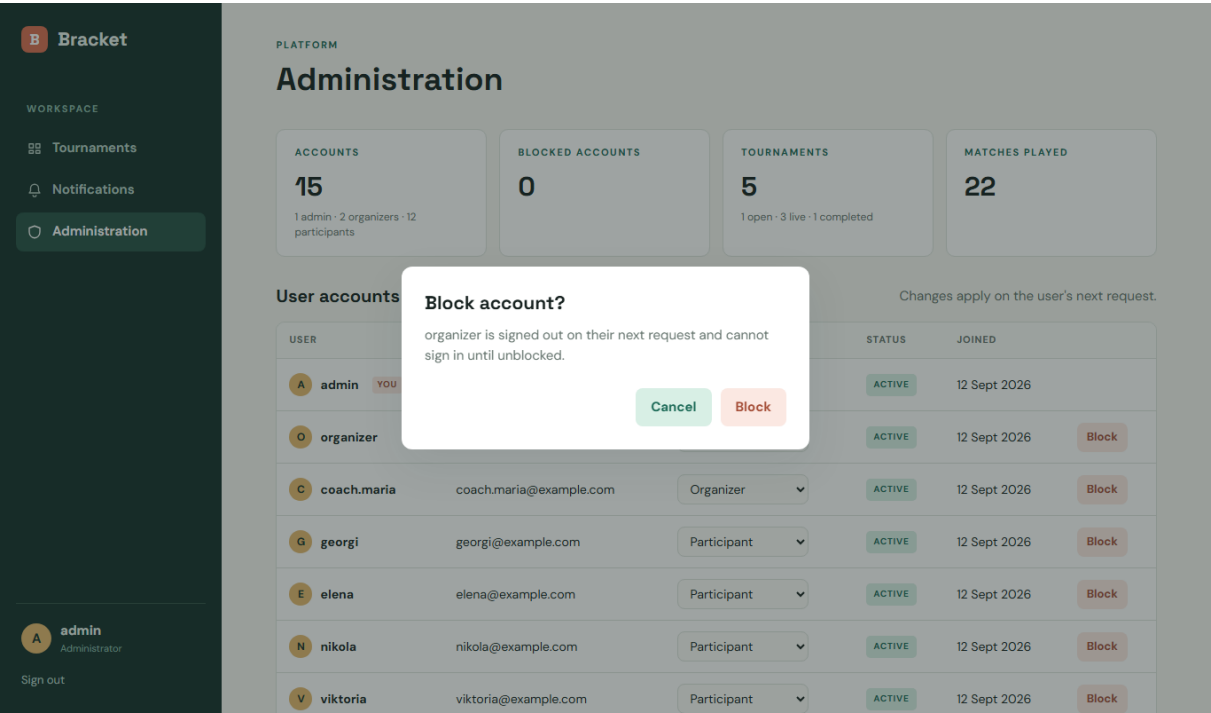
Фигура 27. Списък с турнири от гледна точка на участник



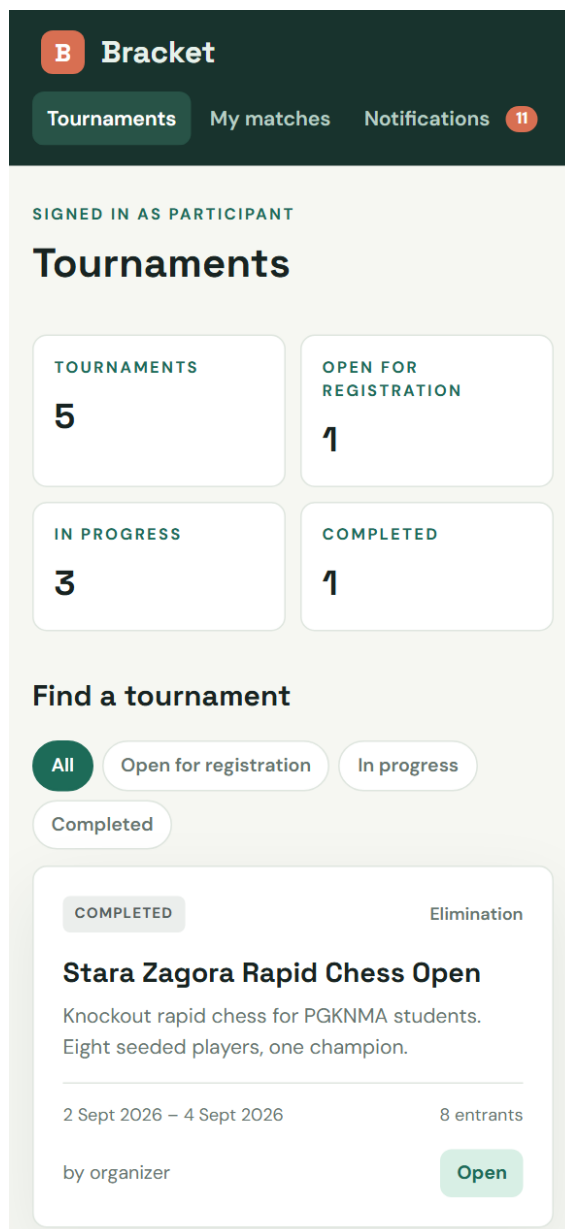
Фигура 28. Записване на участник в турнир с отворена регистрация



Фигура 29. Известия на участник



Фигура 30. Потвърждение преди блокиране на профил



Фигура 31. Изглед на телефон

Приложение В. Стартиране на системата

В.1. Изисквания

- JDK 17 или по-нова версия;
- MySQL Server 8.0;
- Node.js 18 или по-нова версия – за скрипта за стартиране, за статичния сървър на клиента и за сквозните проверки.

В.2. Стартиране с една команда

- 1) Копирайте файла `application-local.properties.example` като `application-local.properties` и попълнете потребителското име и паролата за MySQL. Базата данни `tournament_platform` и таблиците се създават автоматично при първото стартиране.
- 2) Изпълнете една команда в папката на проекта. За представяне добавете `--demo`, за да се създадат демонстрационните данни – те се създават само в база данни без турнири.
- 3) Отворете адреса `http://localhost:5173` в браузъра. Комбинацията `Ctrl+C` спира и сървъра, и клиента.

```
# сървър и клиент с една команда (PowerShell, в папката на проекта)
node dev.js

# същото, но с демонстрационни данни
node dev.js --demo
```

Скриптът `dev.js` е написан без външни зависимости: при липса на изпълним архив той го изгражда еднократно с Maven Wrapper, намира Java чрез `JAVA_HOME`, чрез `PATH` или в обичайните директории за инсталация, стартира сървъра като отделен процес и обслужва клиента в същия процес на Node.js. Изходът на двете части е с представки `[backend]` и `[runner]`. Ако на порт 8080 вече отговаря сървър, скриптът го използва и стартира само клиента, а променливата `PORT` задава друг порт за клиента. Същите команди са достъпни и като `npm start` и `npm run demo`, без да е необходимо инсталиране на пакети.

В.3. Стартиране на двете части поотделно

```
# сървър с демонстрационни данни
$env:DEMO_DATA = "true"
```

```
.\mvnw.cmd spring-boot:run

# клиент, в нов терминал
cd frontend
node server.js
```

В.4. Демонстрационни профили

Демонстрационните данни съдържат четири турнира – приключил турнир по шах с 8 участници, турнир по електронни спортове с 6 отбора в ход, групово лига по тенис на маса и турнир с отворена регистрация – и профилите от Таблица 13. Всички профили използват паролата Demo12345, затова демонстрационните данни не трябва да се включват в реална среда.

Таблица 13. Демонстрационни профили

Потребителско име	Роля
admin	Администратор
organizer, coach.maria	Организатор
georgi, elena, nikola, viktorina, stefan, kalina, martin, yoana, petar, desislava, ivan, radostina	Участник

В.5. Изпълнение на тестовете

```
# автоматизирани тестове
.\mvnw.cmd test

# сквозна проверка на работещо приложение с празна база и демонстрационни данни
$env:API_URL = "http://localhost:8080/api"
node e2e/api-smoke.js
```